Generative Actor-Critic with Soft Bridge Policies

Ke He
Guangzhou University
ke.he@gzhu.edu.cn

Le He*
Guangzhou University
hele@e.gzhu.edu.cn

Shunpu Tang
Zhejiang University
tangshunpu@zju.edu.cn

Yafei Wang
Southeast University
wangyf@seu.edu.cn

Lisheng Fan
Guangzhou University
lsfan@gzhu.edu.cn

Abstract

Expressive generative policies such as diffusion and flow models are appealing for maximum-entropy (MaxEnt) online reinforcement learning because of their ability to model multimodal and highly non-Gaussian action distributions. However, training effective soft generative policies faces two obstacles that often arise together. First, marginal action densities are often unavailable, so existing methods typically rely on entropy bounds, heuristic proxies or approximations. Second, iterative shared-parameter samplers raise inference cost and require backpropagation through time over repeated network evaluations, increasing memory cost and potentially destabilizing policy optimization. These obstacles motivate us to seek a generative actor-critic method that exposes a tractable soft objective while generating each action in just one network evaluation. To this end, we propose *soft generative actor-critic* (`SoftGAC`), whose actor is a single stochastic feedforward network with internal Gaussian bridge layers. This explicit bridge structure allows us to lift the soft objective to path space, yielding an analytical path-KL regularizer against a high-entropy reference process for both actor improvement and the soft Bellman bootstrap, without estimating the endpoint entropy. Crucially, the actor generates each action in only one network evaluation under a total parameter budget comparable to strong baselines, substantially reducing inference cost relative to the existing iterative diffusion and flow policies. It also avoids long unrolled backpropagation through time. Across challenging continuous-control benchmarks, our method achieves competitive or superior returns compared with strong diffusion and flow policies, demonstrating a clear advantage in the compute-return tradeoff.

Code: `https://github.com/skypitcher/soft_gac_jax`

1 Introduction

Maximum-entropy (MaxEnt) online reinforcement learning (RL) has become a central approach for off-policy continuous-control because it combines value improvement with explicit stochasticity (Levine, 2018; Oh et al., 2025; Haarnoja et al., 2018a,b). In soft actor-critic (SAC) (Haarnoja et al., 2018b), this stochasticity is not only an exploration device. It is part of the optimization objective, where the actor is encouraged to choose high-value actions while maintaining high entropy (Haarnoja et al., 2018a). This principle is simple and effective when the policy has a tractable density, but the usual Gaussian policy class can be too restrictive for complex control problems that may require multimodal or highly non-Gaussian action distributions (Nauman et al., 2024; Chao et al., 2024; Li et al., 2024; Park et al., 2025).

*Corresponding author: Le He (`hele@e.gzhu.edu.cn`).

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

Expressive generative policies offer a natural way to expand the actor class (Ho et al., 2020; Song et al., 2021; Lipman et al., 2022; McAllister et al., 2025; Ren et al., 2025). Diffusion, score-based and flow-style policies can represent rich action distributions and have recently become attractive for reinforcement learning (Psenka et al., 2024; Ding et al., 2024; Yu et al., 2025; Lv et al., 2025; Zhang et al., 2025). Their appeal, however, creates a tension with MaxEnt RL. The soft actor update is an endpoint-density objective that requires policy entropy or an equivalent density-based regularizer. For many generative actors, the marginal action density is unavailable or expensive to evaluate because the action is obtained after marginalizing over latent variables or generation paths. Existing methods therefore often introduce entropy bounds (Celik et al., 2025), entropy estimators and approximations (Wang et al., 2024; Ding et al., 2024), noise-augmented path likelihoods (Zhang et al., 2026) or Wasserstein-geometric proxies (Lv et al., 2026; Zhu et al., 2026), which make the connection to MaxEnt RL indirect.

A second difficulty comes from how many generative policies spend computation. Diffusion or flow-style samplers with many denoising or refinement steps usually reuse the same network across time (Ren et al., 2025; Celik et al., 2025; Psenka et al., 2024; Ding et al., 2024). This raises inference cost at deployment and it also makes the number of function evaluations (NFE) a training-time cost, because the actor update differentiates $Q(s, a)$ through the sampled action and every refinement step that produced it (Nguyen and Yoo, 2026). Increasing NFE therefore increases BPTT depth, activation memory and actor-update wall-clock time (Frans et al., 2025). It can also lengthen gradient paths through repeated uses of shared parameters, which may destabilize actor optimization. As a result, high-NFE policies may achieve stronger performance, but the added optimization burden often limits their marginal gains (Ding et al., 2024). Conversely, reducing NFE lowers both training and inference cost, but can cause a sharp performance drop (Park et al., 2025; Celik et al., 2025).

These two obstacles motivate us to seek a different design principle. Specifically, we seek a generative actor-critic design that retains the expressiveness of stochastic latent generation while still exposing a tractable soft objective and generating each action with only one network evaluation. Our goal is not to trade policy performance for cheaper action generation, nor to make soft regularization rely on entropy bounds, heuristic proxies or endpoint-entropy estimates. Instead, we ask whether careful actor-critic design can make an expressive generative policy both faster and stronger while still providing an analytical soft regularizer. We aim to match or exceed the returns of strong iterative diffusion and flow policies using only one network evaluation per action under a comparable total actor parameter budget. To achieve this goal, our key idea is to model the internal stochastic computation of a single feedforward actor as an explicit latent path, rather than treating the actor only through its terminal action distribution. This path-law view replaces endpoint entropy with path-wise Kullback–Leibler (KL) regularization relative to a high-entropy reference process. We show that, with unrestricted path laws and a uniform reference action marginal, the resulting objective recovers the same optimal action distribution as standard MaxEnt policy improvement while additionally regularizing how actions are generated.

We instantiate this idea as *soft generative actor-critic* (`SoftGAC`), an off-policy actor-critic method with soft bridge policies, as illustrated in Figure 1. Its actor is a single stochastic feedforward network whose lightweight internal Gaussian transitions form a bridge in pre-tanh latent space from a fixed base latent to a terminal action latent. These explicit transitions make the local KL terms analytically tractable, allowing us to evaluate the finite-step regularizer as transition control energy along each sampled path. The same forward pass thus generates an action and computes its control energy, which the actor update directly trades off against critic value.

Our contributions are three-fold. **(i)** We formulate a path-space soft objective for generative actors, show that it contains the endpoint KL regularizer used by MaxEnt RL as a marginal component and characterize both its unrestricted endpoint-equivalent optimum and the effect of using a practical fixed actor base. **(ii)** We design a soft bridge policy that uses a short sequence of local Gaussian transitions in pre-tanh latent space, making the path-wise regularizer exactly computable as finite-step control energy and enabling a direct actor update with action generation by a single feedforward pass under a comparable actor parameter budget. **(iii)** We demonstrate on challenging continuous-control benchmarks that `SoftGAC` improves the compute-return tradeoff by pairing strong returns with low one-pass action-generation cost and a parameter budget comparable to strong actor baselines.

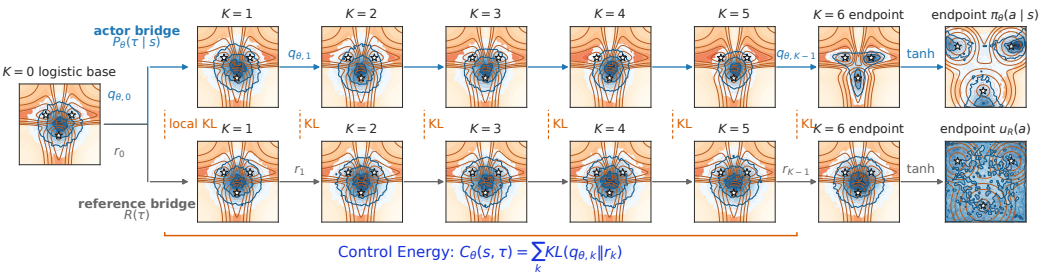


Figure 1: Overview of the proposed soft bridge policy. Local KL terms compare actor and reference transitions and sum to the sampled control energy. The terminal latent is mapped through $\tanh$ and optimized by the critic. Appendix C provides a 2D bridge visualization.

2 Preliminaries

Maximum-entropy reinforcement learning. We consider a discounted Markov decision process with state space $\mathcal{S}$, bounded continuous action space $\mathcal{A} \subset \mathbb{R}^{d_a}$, reward $r(s,a)$, transition kernel $p(s' \mid s,a)$, discount factor $\gamma \in (0,1)$ and policy $\pi(a \mid s)$. Maximum-entropy reinforcement learning augments the expected return with an entropy bonus,

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \left(r(s_t,a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t)) \right) \right], \tag{1}$$

where $\alpha > 0$ controls the strength of the soft regularizer. In an off-policy actor-critic algorithm, a critic estimates a soft value landscape and the actor is improved by increasing value while preserving stochasticity. When $\mathcal{A}$ is bounded and u denotes the uniform action law, the standard per-state soft actor improvement step instantiated by SAC (Haarnoja et al., 2018b) can be written as

$$\mathcal{J}_{\mathrm{SAC}}(\pi \mid s) = \mathbb{E}_{a \sim \pi(\cdot|s)}[Q(s,a)] - \alpha D_{\mathrm{KL}}(\pi(\cdot \mid s) \,\|\, u) + \text{const.} \tag{2}$$

This form will be useful below because it separates the value term from a regularizer that keeps the endpoint action distribution close to a high-entropy reference. Explicit-density policies such as standard SAC can evaluate this objective because the policy density is available. The difficulty begins when the policy is a generative sampler whose endpoint density is not directly exposed.

Generative actors as path laws. Many recent generative policies naturally fit a common path-law view. A diffusion actor (Ho et al., 2020; Li et al., 2024; Celik et al., 2025) starts from base noise and samples a reverse denoising chain,

$$y_{k-1} \sim p_\theta(y_{k-1} \mid y_k, s), \qquad y_K \sim \mathcal{N}(0, I), \tag{3}$$

while a flow or flow-matching actor (Lipman et al., 2022; McAllister et al., 2025; Zhang et al., 2025) evolves base noise through a learned velocity field,

$$\frac{dy_t}{dt} = v_\theta(y_t, t, s), \qquad y_0 \sim p_0. \tag{4}$$

Beyond multi-step implicit generators, one-shot implicit actors form a degenerate short-path case, where $z_0 \sim p_0$ is mapped by a neural generator to a terminal latent $z_1 = f_\theta(z_0, s)$ (Zhu et al., 2026). These actors all generate an action by following a latent path from base noise to a terminal latent state. For the rest of the paper, we use the forward ordering $\tau = (z_0, z_1, \ldots, z_K)$, where z_0 denotes the base latent and z_K denotes the terminal action latent. The actor samples this path from a path law $P_\theta(d\tau \mid s)$ and produces an action through a terminal map $a = T(z_K)$. In this paper, T maps the terminal latent state to a bounded action through a tanh transform, but we only need the induced endpoint law $\pi_\theta(\cdot \mid s) = (T \circ z_K)_\# P_\theta(\cdot \mid s)$, where $\#$ denotes the pushforward of a probability law. Equivalently, this is the distribution of $T(z_K)$ when $\tau \sim P_\theta(\cdot \mid s)$. Such an actor can be easy to sample from while still having an intractable marginal action density. The density of $\pi_\theta(a \mid s)$ may require integrating over all latent paths that terminate at the same action, which is generally unavailable for implicit generators and costly for multi-step diffusion or flow samplers.

3 Maximum-Entropy Reinforcement Learning in Path Space

3.1 Path-Space Objective

When the terminal action density is unavailable, a natural approach is to lift MaxEnt RL from the terminal action distribution to the full generation path. Since a generative actor samples a latent path τ before producing a terminal action, we define the soft objective over path laws. Let $R(d\tau)$ be a high-entropy reference path law in the same latent space and let u_R be its terminal action law induced by $T(z_K)$. For a fixed state s and critic Q, we consider

$$\mathcal{J}_{\mathrm{path}}(P_\theta \mid s) = \mathbb{E}_{\tau\sim P_\theta(\cdot|s)}\left[Q(s, T(z_K))\right] - \alpha D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s) \,\|\, R\right). \tag{5}$$

The value term still evaluates only the terminal action. The regularizer, however, now compares the full generation path to a high-entropy reference process. This path-space objective is useful only if it remains connected to the endpoint MaxEnt objective. We show this connection by decomposing the path KL into a terminal-action term and a conditional path term.

Proposition 1 (Path-space lift of the endpoint KL regularizer)**.** *Let $a = T(z_K)$ be the terminal action. Suppose the actor path law and the reference path law admit disintegrations*

$$P_\theta(d\tau \mid s) = \pi_\theta(da \mid s)\, P_\theta(d\tau \mid s, a), \qquad R(d\tau) = u_R(da)\, R(d\tau \mid a). \tag{6}$$

Then

$$D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s) \,\|\, R\right) = D_{\mathrm{KL}}\left(\pi_\theta(\cdot \mid s) \,\|\, u_R\right) + \mathbb{E}_{a\sim\pi_\theta}\left[D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s, a) \,\|\, R(\cdot \mid a)\right)\right]. \tag{7}$$

The proof is a direct chain-rule decomposition of relative entropy and appears in Appendix A.1. This decomposition shows that the path KL contains the endpoint KL regularizer as its marginal component. When u_R is uniform over the bounded action space, the first term in Eq. (7) is exactly the uniform-prior endpoint regularizer in Eq. (2) up to a constant. The second term is the additional structure introduced by the path-space lift. It regularizes how the actor generates an action, not only which terminal action distribution it induces. Thus the path objective is stricter than endpoint entropy while still preserving the endpoint regularizer of MaxEnt RL.

3.2 Unrestricted Soft-Optimal Bridge

We next ask whether this stricter objective changes the ideal soft-optimal endpoint action distribution. If we optimize over all path laws that are absolutely continuous with respect to the reference, the answer is no in the ideal uniform-reference case.

Theorem 1 (Unrestricted soft-optimal bridge)**.** *For fixed s, Q and $\alpha > 0$, consider the unrestricted optimization of Eq.* (5) *over path laws P with $P \ll R$. Assume*

$$Z_Q(s) = \mathbb{E}_{\tau\sim R}\left[\exp\left(Q(s, T(z_K))/\alpha\right)\right] < \infty. \tag{8}$$

The unique optimum is the value-tilted reference path law

$$\frac{dP_Q^\star}{dR}(\tau \mid s) = \frac{\exp\left(Q(s, T(z_K))/\alpha\right)}{Z_Q(s)}. \tag{9}$$

Its terminal action law is

$$\pi_Q^\star(a \mid s) = \frac{u_R(a)\exp\left(Q(s,a)/\alpha\right)}{\int_{\mathcal{A}} u_R(b)\exp\left(Q(s,b)/\alpha\right) db}. \tag{10}$$

The proof follows from the Gibbs variational identity and appears in Appendix A.2. Since the value tilt in Eq. (9) depends only on the terminal action, the optimal conditional path law given that action remains the reference conditional bridge $R(d\tau \mid a)$. Value changes the endpoint marginal, while the reference controls how each endpoint is reached. When u_R is uniform, Eq. (10) recovers the usual Boltzmann endpoint policy proportional to $\exp(Q(s,a)/\alpha)$. The unrestricted path-space objective is therefore *endpoint-equivalent* to the MaxEnt actor update in the ideal reference limit, but it is more selective about the generation path.

3.3 Fixed-Base Bridge Choice

The unrestricted optimum gives a clean conceptual target, but every implementable generative actor also needs to specify how its latent path starts. A diffusion sampler, flow sampler or one-shot generator all begin from some base law. This base law is a practical modeling choice. If the actor base differs from the reference base, the path KL gains an initial-law term before the remaining conditional path is compared to the reference. For a fixed actor base, this initial term is constant with respect to the conditional path law after z_0. Thus the base choice does not change the variational problem solved after conditioning on the base sample. In finite implementations, it mainly acts as an inductive bias by shaping the initial latents from which the rest of the generative path is drawn. To characterize this effect, let the reference path law disintegrate as

$$R(d\tau) = r_0(dz_0)\, R(d\tau_{1:K} \mid z_0), \tag{11}$$

and fix an actor base law p_0 with $p_0 \ll r_0$. We optimize the same path objective as Eq. (5), but restrict the actor path law to satisfy $P_0 = p_0$. The value tilt still points toward the unrestricted soft optimum in Theorem 1, while the actor can only search within the selected fixed-base family. The best attainable solution is therefore the constrained optimum below.

Theorem 2 (Fixed-base bridge optimum)**.** *Define*

$$Z_Q(s, z_0) = \mathbb{E}_{\tau \sim R(\cdot \mid z_0)} \left[\exp\left(Q(s, T(z_K))/\alpha\right) \right], \qquad \bar{Z}_Q(s) = \mathbb{E}_{z_0 \sim r_0} \left[Z_Q(s, z_0) \right]. \tag{12}$$

Among all path laws with initial marginal p_0, the unique optimum is

$$P^\star_{Q,p_0}(d\tau \mid s) = p_0(dz_0)\, \frac{\exp\left(Q(s, T(z_K))/\alpha\right)}{Z_Q(s, z_0)} R(d\tau_{1:K} \mid z_0). \tag{13}$$

Its objective value is

$$\alpha \mathbb{E}_{z_0 \sim p_0} \left[\log Z_Q(s, z_0) \right] - \alpha D_{\mathrm{KL}} \left(p_0 \,\|\, r_0 \right). \tag{14}$$

The proof applies the Gibbs variational identity conditionally on the fixed base latent and appears in Appendix A.3. The initial KL term is independent of the conditional optimizer after z_0. Thus using a different fixed base is not a heuristic break from the objective. It changes the full path objective by a constant and changes the finite actor's inductive bias through the initial samples. When the fixed actor base matches the value-tilted initial marginal of the unrestricted bridge, this constrained optimum coincides with the unrestricted one. Otherwise, the actor does not reweight the initial latent distribution in a state-dependent way, and the corollary quantifies the resulting gap.

Corollary 1 (Base-constraint gap controls endpoint deviation)**.** *Under the assumptions of Theorem 2, the loss relative to the unrestricted optimum is*

$$\Delta(s) = \alpha \left(\log \bar{Z}_Q(s) - \mathbb{E}_{z_0 \sim p_0} \left[\log Z_Q(s, z_0) \right] + D_{\mathrm{KL}} \left(p_0 \,\|\, r_0 \right) \right) \geq 0. \tag{15}$$

Equality holds if and only if p_0 equals the initial marginal of the unrestricted tilted bridge. In the special case $p_0 = r_0$, this reduces to $Z_Q(s, z_0)$ being constant for p_0-almost every z_0. Moreover,

$$D_{\mathrm{KL}} \left(P^\star_{Q,p_0}(\cdot \mid s) \,\|\, P^\star_Q(\cdot \mid s) \right) = \frac{\Delta(s)}{\alpha}, \tag{16}$$

and the induced endpoint deviation obeys

$$D_{\mathrm{KL}} \left(\pi^\star_{Q,p_0}(\cdot \mid s) \,\|\, \pi^\star_Q(\cdot \mid s) \right) \leq \frac{\Delta(s)}{\alpha}. \tag{17}$$

The proof is given in Appendix A.4. The unrestricted optimum can tilt the initial latent law by $Z_Q(s, z_0)$, which would require a state-dependent base sampler before any bridge transition. A fixed-base actor avoids this sampler. Its cost is $\Delta(s)$, and we show that this same cost controls the endpoint deviation. The bound is small unless $Z_Q(s, z_0)$ varies sharply across likely base latents, meaning some base regions are much better positioned to reach high-value actions than others. In the regime we target, typical base samples can be transported to useful endpoints. The base choice then acts mainly as a practical inductive bias, while the bridge transitions perform value-guided transport. For a finite-step implementation, the reference terminal law is u_R rather than the ideal uniform action law. The endpoint-MaxEnt connection is exact with respect to u_R and recovers the uniform-prior view when $u_R = u$. What `SoftGAC` optimizes is more concrete. For the chosen actor-reference pair, the finite-step path KL is the exact regularizer up to the fixed initial-base constant. We now need an architecture that exposes this KL from sampled paths.

4 Path-Regularized Generative Actor-Critic

The preceding section turns soft policy improvement into a path-law optimization problem with a fixed base distribution. This leaves an architectural question. How can the actor expose the path likelihood needed by the regularizer while remaining cheap to sample? We answer this question with `SoftGAC`, a path-regularized actor-critic that combines a short bridge actor with an off-policy soft critic. The bridge actor makes the finite-step path KL factor into local Gaussian terms. These terms will become the sampled control energy used by the actor and critic updates.

4.1 Soft Bridge Policies

A soft bridge policy is a finite Markov chain in pre-tanh latent space. It starts from a fixed base law and applies K lightweight Gaussian residual transitions before mapping the terminal latent state to the bounded action. This gives an explicit path law and avoids the long shared-parameter sampler used by high-NFE iterative policies. We write its transition law as

$$z_0 \sim p_0, \qquad q_{\theta,k}(z_{k+1} \mid z_k, s) = \mathcal{N}\left(z_k + h\, u_{\theta,k}(s, z_k), 2h \operatorname{diag}(\sigma^2_{\theta,k}(s, z_k))\right), \tag{18}$$

for $k = 0, \dots, K-1$ and $h = 1/K$. The terminal action is $a = T(z_K) = \tanh(z_K)$ after the usual affine rescaling to the environment action bounds. Each transition block has its own parameters and communicates with the next block only through the action-dimensional latent state z_k. Thus the forward pass is a stochastic generative process, but it is still a fixed-depth one-pass network rather than an iterative sampler that reuses the same refinement model many times. During actor updates, gradients pass through this short sequence of transition blocks instead of a long BPTT graph over repeated uses of a shared sampler.

The reference bridge plays the role of a high-entropy action prior. In normalized action space $(-1, 1)^{d_a}$, the pre-tanh density whose tanh image is uniform is

$$q_{\mathrm{ref}}(z) = \frac{1}{2^{d_a}} \prod_{i=1}^{d_a} \operatorname{sech}^2(z_i), \qquad \nabla_z \log q_{\mathrm{ref}}(z) = -2 \tanh(z). \tag{19}$$

For the main implementation, we set the actor base law to this reference density, $p_0 = q_{\mathrm{ref}}$. This keeps the full path KL free of an initial constant and gives the cleanest decomposition below. Other fixed bases are also valid. They add a parameter-independent initial KL to the full objective and mainly act as an implementation-level inductive bias through the base samples. The finite-step reference starts from the logistic reference base and uses an Euler Gaussian kernel based on the score above,

$$r_k(z_{k+1} \mid z_k) = \mathcal{N}\left(z_k - 2h \tanh(z_k), 2hI\right). \tag{20}$$

The continuous reference diffusion is stationary when initialized from q_{ref}, so its tanh image is uniform in action space at every time. The finite-step reference is the object used by the algorithm. The path KL below is exact for this bridge pair, while Appendix D quantifies its endpoint bias.

Lemma 1 (Finite-step control-energy decomposition)**.** *Let*

$$P_\theta(d\tau \mid s) = p_0(dz_0) \prod_{k=0}^{K-1} q_{\theta,k}(dz_{k+1} \mid z_k, s), \qquad R(d\tau) = r_0(dz_0) \prod_{k=0}^{K-1} r_k(dz_{k+1} \mid z_k). \tag{21}$$

Then

$$D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s) \,\|\, R\right) = D_{\mathrm{KL}}\left(p_0 \,\|\, r_0\right) + \sum_{k=0}^{K-1} \mathbb{E}_{\tau \sim P_\theta}\left[D_{\mathrm{KL}}\left(q_{\theta,k}(\cdot \mid z_k, s) \,\|\, r_k(\cdot \mid z_k)\right)\right]. \tag{22}$$

For Gaussian local kernels, define $\mathcal{C}_\theta(s, \tau)$ as the sum of the local Gaussian KL terms. Then

$$D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s) \,\|\, R\right) = D_{\mathrm{KL}}\left(p_0 \,\|\, r_0\right) + \mathbb{E}_{\tau \sim P_\theta(\cdot|s)}\left[\mathcal{C}_\theta(s, \tau)\right]. \tag{23}$$

The proof is a factorization of the Markov path likelihood ratio and appears in Appendix A.5. When $p_0 = r_0$, the initial KL vanishes. When p_0 is a different fixed base, this initial term is constant with respect to the actor transitions, so actor training can use the same sampled transition control energy. For the residual actor in Eq. (18) and the reference in Eq. (20), the local control cost is

$$\mathcal{C}_k(s, z_k) = \frac{1}{2} \sum_{i=1}^{d_a} \left[\sigma^2_{\theta,k,i}(s, z_k) + \frac{\left(\mu_{\theta,k,i}(s, z_k) - \mu_{R,k,i}(z_k)\right)^2}{2h} - 1 - \log \sigma^2_{\theta,k,i}(s, z_k)\right], \tag{24}$$

where $\mu_{\theta,k}(s, z_k) = z_k + h\, u_{\theta,k}(s, z_k)$ and $\mu_{R,k}(z_k) = z_k - 2h \tanh(z_k)$. The sampled path cost is $\mathcal{C}_\theta(s, \tau) = \sum_{k=0}^{K-1} \mathcal{C}_k(s, z_k)$. This quantity is the finite-step transition control energy of the bridge path, and its expectation is the actor-reference path KL up to the fixed initial-base constant. The soft regularizer in `SoftGAC` is therefore an analytical path-wise relative entropy rather than an estimate, bound or proxy for endpoint action entropy. The next subsection uses this finite-step path regularizer inside an off-policy actor-critic update.

4.2 Off-Policy Actor-Critic Training

We train this bridge actor with an off-policy critic in `SoftGAC`. For a fixed bridge policy, define the path-regularized soft value

$$V^\pi(s) = \mathbb{E}_{\tau \sim P_\theta(\cdot|s)} \left[Q^\pi(s, T(z_K)) - \alpha\, \mathcal{C}_\theta(s, \tau) \right]. \tag{25}$$

The corresponding Bellman equation is

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \left[V^\pi(s') \right]. \tag{26}$$

Thus the only change from a standard soft actor-critic update is that endpoint entropy is replaced by sampled bridge control energy. Given a replay batch (s, a, r, s', d), we sample a next-state bridge path $\tau' \sim P_{\bar{\theta}}(\cdot \mid s')$ from the target actor and form the soft bootstrap scalar

$$y = r + \gamma(1 - d) \left[Q^{\min}_{\bar{\phi}}(s', T(z'_K)) - \alpha\, \mathcal{C}_{\bar{\theta}}(s', \tau') \right]. \tag{27}$$

The critic can be any off-policy estimator trained toward this target. Our implementation uses a twin categorical critic with a fixed support $\mathcal{Z} = \{v_1, \ldots, v_M\}$ and a CrossQ-style update (Bellemare et al., 2017; Bhatt et al., 2024; Celik et al., 2025). For each critic head, the target distribution is obtained by shifting the support by the sampled soft bootstrap and projecting back to $\mathcal{Z}$ with the usual categorical projection. This critic choice is an implementation detail rather than a requirement of the bridge objective. The actor is updated by sampling current-state paths $\tau \sim P_\theta(\cdot \mid s)$ and minimizing

$$\mathcal{L}_{\text{actor}}(\theta) = \mathbb{E}_{s \sim \mathcal{D}} \mathbb{E}_{\tau \sim P_\theta(\cdot|s)} \left[\alpha\, \mathcal{C}_\theta(s, \tau) - Q^{\min}_\phi(s, T(z_K)) \right]. \tag{28}$$

At a fixed state, this sampled objective has a precise projection interpretation. It moves the transition actor toward the ideal soft-optimal bridge, while the chosen fixed base determines the practical actor family being searched.

Proposition 2 (Actor update as restricted projection to the ideal bridge)**.** *Let $P^\star_Q$ be the unrestricted soft-optimal bridge in Theorem 1. For any finite-step bridge actor with fixed base law $p_0 \ll r_0$ that satisfies Lemma 1,*

$$\mathcal{L}_{\text{actor}}(\theta \mid s) = \mathbb{E}_{\tau \sim P_\theta(\cdot|s)} \left[\alpha\, \mathcal{C}_\theta(s, \tau) - Q(s, T(z_K)) \right] \tag{29}$$

satisfies

$$\mathcal{L}_{\text{actor}}(\theta \mid s) = \alpha D_{\text{KL}} \left(P_\theta(\cdot \mid s) \,\|\, P^\star_Q(\cdot \mid s) \right) - \alpha \log Z_Q(s) - \alpha D_{\text{KL}} \left(p_0 \,\|\, r_0 \right). \tag{30}$$

Consequently, minimizing this loss over the unrestricted fixed-base path-law family has global minimizer $P^\star_{Q,p_0}$ from Theorem 2.

The proof appears in Appendix A.6. The global-minimizer statement is a property of the unrestricted fixed-base path-law family. The finite neural Gaussian Markov actor optimized by SGD is a restricted parameterization, so the result should not be read as a global optimization guarantee for the implemented network. Equation (30) is nevertheless useful algorithmically. Once the base law is fixed, the final two terms are constants with respect to the transition actor parameters, and the sampled actor loss follows a reverse-KL projection direction toward the ideal bridge. In the implemented neural family, the loss defines a tractable finite-step objective whose attainable solution is limited by parameterization and optimization.

In addition, we tune the temperature with a SAC-style dual update on the control-energy budget. Let $\mathcal{C}_{\text{target}} = \rho_{\text{ctrl}} K d_a$ be a per-step, per-dimension heuristic target cost. Appendix E.4 gives the scale intuition behind this choice. The temperature objective is

$$\mathcal{L}_\alpha = \mathbb{E} \left[\alpha \left(\mathcal{C}_{\text{target}} - \mathcal{C}_\theta(s, \tau) \right) \right], \qquad \alpha = \exp(\log \alpha). \tag{31}$$

This keeps the average bridge deviation from the high-entropy reference near a chosen budget. Action generation remains fixed-cost with exactly K local transition blocks. The complete training loop is given in Algorithm 1 in Appendix B.

5 Experiments

Experimental setup. We evaluate SoftGAC on challenging high-dimensional continuous-control tasks from the DeepMind Control Suite (DMC) (Tassa et al., 2018; Tunyasuvunakool et al., 2020) and HumanoidBench (Sferrazza et al., 2024). We measure both return and the compute-return tradeoff. Generative baselines include FLAC (Lv et al., 2026), DIME (Celik et al., 2025), FlowRL (Lv et al., 2025), QSM (Psenka et al., 2024), QVPO (Ding et al., 2024), DACER (Wang et al., 2024), MaxEntDP (Dong et al., 2025), SDAC (Ma et al., 2025) and a conditional normalizing-flow actor (Ghugare and Eysenbach, 2025). CrossQ-SAC (Bhatt et al., 2024) provides the diagonal-Gaussian baseline. We report interquartile mean (IQM) (Agarwal et al., 2021) over 8 random seeds with 95% bootstrap confidence intervals. Ablations isolate the contribution of the path-space regularizer.

We re-implement all baselines in a single codebase under the framework of StableBaselines3 (Raffin et al., 2021), and we follow the official implementations whenever available. All methods share the same infrastructure and main critic update. We adjust actor sizes to obtain comparable parameter budgets, while the reported latency measures the complete action-generation computation for each actor. Our SoftGAC actor uses $K = 6$ bridge transitions, each represented by a lightweight single-layer MLP with mean and variance heads. For deterministic evaluation and deployment, our SoftGAC actor follows the mean bridge path, bypassing the variance heads and omitting the control-energy computation. Appendices E and F give the full implementation details, hyperparameters, experimental setup, extended results and ablations.

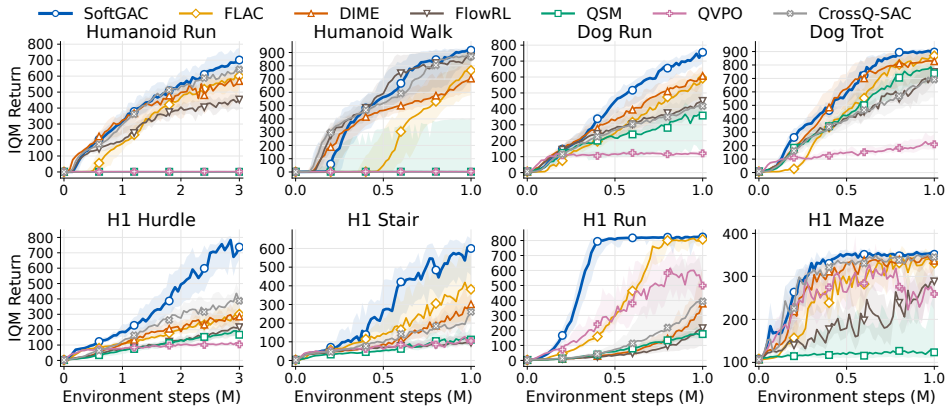


Figure 2: IQM learning curves on the 8 hard control tasks.

Performance. Figure 2 shows that SoftGAC gives the strongest overall performance on the hard tasks. The gains are largest on high-dimensional, long-horizon locomotion tasks such as Humanoid Run, Dog Run, H1 Hurdle and H1 Stair, where SoftGAC improves over both generative baselines and the CrossQ-SAC Gaussian baseline. On tasks where CrossQ-SAC is already strong, such as Humanoid Walk and H1 Maze, SoftGAC remains competitive or better while retaining a richer generative actor. These results suggest that the soft bridge actor improves sample efficiency and final performance, rather than only increasing expressiveness at convergence.

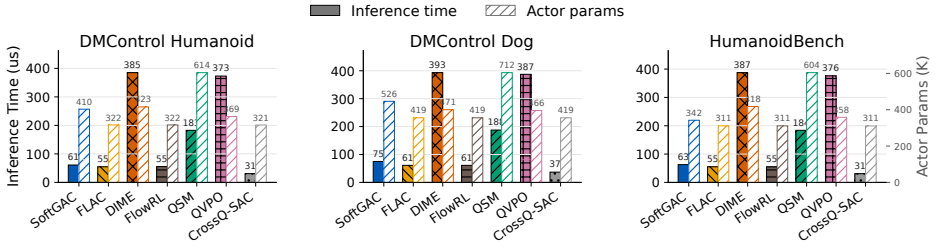


Figure 3: Per-action inference time and actor parameter count measured on an Apple M3 Pro CPU.

Inference cost. Figure 3 shows that this return gain does not come from unrolled sampler evaluations. CrossQ-SAC has the cheapest unimodal Gaussian actor. `SoftGAC` uses about 61–75 μs per action on the CPU benchmark, which places it in the same low-latency range as one-step flow baselines and far below high-NFE diffusion baselines. This matters because actor sampling is used both during environment interaction and inside actor-critic updates. The soft bridge policy generates each action with one sampled pass through its bridge blocks and keeps the actor parameter budget comparable, while still offering superior performance. This explains the improved compute-return tradeoff observed in Figures 2 and 3.

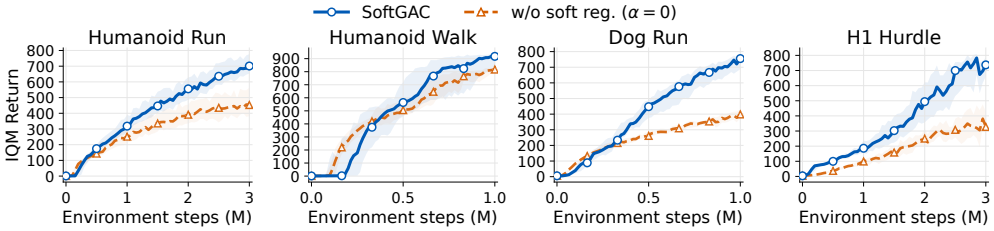


Figure 4: Ablation study of the soft regularizer.

Soft regularization. Figure 4 isolates the path-space soft regularizer by setting $\alpha = 0$ while keeping the same actor and critic. Removing this term consistently hurts Humanoid Run, Humanoid Walk, Dog Run and H1 Hurdle, which suggests that the gains do not come only from the bridge parameterization. The result supports the relative-entropy control energy as a principled soft objective for the one-pass generator, rather than only an exploration heuristic.

6 Related Work and Discussion

Expressive generative policies. Generative policies broaden the actor classes available to continuous-control RL beyond the diagonal Gaussians commonly used in SAC (Haarnoja et al., 2018b). Normalizing flows offer richer distributions while retaining tractable densities. Energy-based normalizing flows integrate policy and value learning into a unified formulation (Chao et al., 2024), while subsequent work demonstrates the competitiveness of normalizing-flow policies across continuous-control settings (Ghugare and Eysenbach, 2025). However, exact action-density evaluation in normalizing-flow policies relies on invertible transformations with tractable Jacobian determinants, limiting the architectural flexibility of the actor. Stein soft actor-critic offers a different construction based on iterative particle transformations and an explicit entropy approximation (Messaoud et al., 2024). Diffusion and flow-matching policies support several reward-driven learning schemes. Diffusion policy gradient learns multimodal behaviors from scratch (Li et al., 2024), Q-score matching uses action gradients of the critic to guide score learning (Psenka et al., 2024) and QVPO constructs a Q-weighted variational policy optimization objective (Ding et al., 2024). Flow matching policy gradients develop on-policy updates from flow-matching losses (McAllister et al., 2025). For pretrained policies, DPPO and ReinFlow study reinforcement-learning fine-tuning of diffusion and flow models (Ren et al., 2025; Zhang et al., 2025). For MaxEnt RL, however, richer actors also raise the two challenges of tractable soft regularization and efficient action generation and optimization.

Soft regularization for implicit policies. The obstacle to soft regularization is that standard soft policy improvement and evaluation require the marginal action log density, which implicit generators generally do not expose. DIME derives a tractable entropy lower bound from diffusion paths and incorporates it into soft policy iteration (Celik et al., 2025), while SAC Flow uses tractable joint likelihoods along noise-augmented generation paths (Zhang et al., 2026). Endpoint entropy and log-density estimates provide another route. DACER fits a Gaussian mixture model to sampled actions and uses its entropy estimate to regulate exploration (Wang et al., 2024). MaxEntDP estimates the Boltzmann score through self-normalized importance sampling and uses Monte Carlo endpoint log-density estimation in its soft Bellman backup (Dong et al., 2025), whereas SDAC combines reverse-sampling reweighted score matching for Boltzmann policy improvement with an additive-noise

log-density approximation for soft evaluation (Ma et al., 2025). Transport and bridge formulations shift the focus from density evaluation to how distributions are connected. FlowRL constrains policy improvement through Wasserstein distance to a replay-derived reference policy (Lv et al., 2025). FLAC is particularly close to our work in motivation, formulating MaxEnt RL through a generalized Schrödinger bridge and kinetic-energy regularization (Lv et al., 2026). In its deterministic flow regime, however, kinetic energy is a geometric proxy that does not by itself imply high terminal entropy. WPPG explicitly supplies the entropy contribution through a heat-flow step after Wasserstein proximal transport, while its W_2 term remains an action-space proximal metric (Zhu et al., 2026). In `SoftGAC`, we directly optimize the actor-reference path KL for the implemented finite-step bridge. Unlike a purely geometric proximity penalty, this KL explicitly contains endpoint KL to the reference action law. For the Gaussian transitions, its trainable part reduces exactly to sampled transition control energy, including both mean and variance terms. We use this same analytical local regularizer in actor improvement and the soft Bellman bootstrap, without fitting the endpoint density or introducing a separate entropy estimator.

Training and inference costs. Tractable regularization alone does not remove the cost of generating and optimizing actions. Many diffusion and flow policies repeatedly evaluate a shared network, so higher NFE directly raises inference cost (Celik et al., 2025; Zhang et al., 2026). When actor updates differentiate $Q(s, a)$ through sampled actions, higher NFE also increases the depth of unrolled backpropagation, activation memory and actor-update cost (Park et al., 2025; Nguyen and Yoo, 2026). Long recursive gradient paths can further destabilize optimization, motivating the velocity-reparameterized sequential models in SAC Flow (Zhang et al., 2026). Simply reducing the number of sampling steps, however, can sacrifice policy quality (Nguyen and Yoo, 2026). FQL instead trains a separate one-step actor with distillation from a flow behavior model, avoiding recursive Q-gradient backpropagation while still requiring teacher sampling during training (Park et al., 2025). One-Step FQL removes the separate distillation stage by learning an average-velocity formulation for direct one-step offline policy generation (Nguyen and Yoo, 2026). Complementary approaches include shortcut models for one-step or few-step generation (Frans et al., 2025) and D3P for state-dependent denoising depth (Yu et al., 2025). `SoftGAC` shares the goal of inexpensive action generation but derives its one-pass actor from the online soft objective itself. We use a compact sequence of step-specific bridge blocks rather than repeated evaluations of a shared sampler, so each sampled forward pass both generates an action and exposes the local transition-control terms. Under a comparable total actor parameter budget, this design addresses soft regularization and actor computation together, without distillation or long recursive backpropagation through a shared sampler.

7 Conclusion and Limitations

We presented `SoftGAC`, a generative actor-critic method that lifts maximum-entropy regularization from endpoint actions to the full latent generation path. This path-space view yields an analytical relative-entropy regularizer, implemented as sampled transition control energy for Gaussian bridges. It also gives a single-pass actor that avoids repeatedly applying a high-NFE shared sampler and long BPTT through that sampler. Across challenging locomotion benchmarks, `SoftGAC` achieves higher or competitive returns while remaining in the low-latency regime of one-pass actors, which supports soft bridge policies as a practical design for efficient generative actor-critic learning.

Compared with Gaussian SAC, our bridge policy introduces additional design choices for depth, transition variance and control budget, although our ablations indicate robustness across the tested settings. Our empirical evaluation also focuses on state-based continuous control in simulation. These considerations motivate future work on bridge architectures that scale effectively with depth and width, as well as image-conditioned policies, integration with world models and real-robot evaluation.

Acknowledgments and Disclosure of Funding

This work was supported by the Guangdong Provincial Key Research and Development Program under Grant 2024B0101040006 and by the National Natural Science Foundation of China (NSFC) under Grants U23A20273, 62271158 and 625B2165. This work was also supported in part by the Google TPU Research Cloud (TRC) program, which provided access to TPU compute resources for the experiments. The authors declare no competing interests.

References

Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv preprint arXiv:1805.00909*, 2018.

Junhyuk Oh, Gregory Farquhar, Iurii Kemaev, Dan A Calian, Matteo Hessel, Luisa Zintgraf, Satinder Singh, Hado Van Hasselt, and David Silver. Discovering state-of-the-art reinforcement learning algorithms. *Nature*, 648(8093):312–319, 2025.

Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018a.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *International Conference on Machine Learning (ICML)*, 2018b.

Michal Nauman, Mateusz Ostaszewski, Krzysztof Jankowski, Piotr Miłoś, and Marek Cygan. Bigger, regularized, optimistic: scaling for compute and sample efficient continuous control. *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

Chen-Hao Chao, Chien Feng, Wei-Fang Sun, Cheng-Kuang Lee, Simon See, and Chun-Yi Lee. Maximum entropy reinforcement learning via energy-based normalizing flow. *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

Zechu Li, Rickmer Krohn, Tao Chen, Anurag Ajay, Pulkit Agrawal, and Georgia Chalvatzaki. Learning multimodal behaviors from scratch with diffusion policy gradient. *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

Seohong Park, Qiyang Li, and Sergey Levine. Flow Q-learning. *International Conference on Machine Learning (ICML)*, 2025.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2020.

Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *International Conference on Learning Representations (ICLR)*, 2021.

Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

David McAllister, Songwei Ge, Brent Yi, Chung Min Kim, Ethan Weber, Hongsuk Choi, Haiwen Feng, and Angjoo Kanazawa. Flow matching policy gradients. *arXiv preprint arXiv:2507.21053*, 2025.

Allen Z Ren, Justin Lidard, Lars L Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. Diffusion policy policy optimization. *International Conference on Learning Representations (ICLR)*, 2025.

Michael Psenka, Alejandro Escontrela, Pieter Abbeel, and Yi Ma. Learning a diffusion model policy from rewards via Q-score matching. *International Conference on Machine Learning (ICML)*, 2024.

Shutong Ding, Ke Hu, Zhenhao Zhang, Kan Ren, Weinan Zhang, Jingyi Yu, Jingya Wang, and Ye Shi. Diffusion-based reinforcement learning via Q-weighted variational policy optimization. *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

Shu-Ang Yu, Feng Gao, Yi Wu, Chao Yu, and Yu Wang. D3P: Dynamic denoising diffusion policy via reinforcement learning. *arXiv preprint arXiv:2508.06804*, 2025.

Lei Lv, Yunfei Li, Yu Luo, Fuchun Sun, Tao Kong, Jiafeng Xu, and Xiao Ma. Flow-based policy for online reinforcement learning. *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2025.

Tonghe Zhang, Chao Yu, Sichang Su, and Yu Wang. Reinflow: Fine-tuning flow matching policy with online reinforcement learning. *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2025.

Onur Celik, Zechu Li, Denis Blessing, Ge Li, Daniel Palenicek, Jan Peters, Georgia Chalvatzaki, and Gerhard Neumann. DIME: Diffusion-based maximum entropy reinforcement learning. *International Conference on Machine Learning (ICML)*, 2025.

Yinuo Wang, Likun Wang, Yuxuan Jiang, Wenjun Zou, Tong Liu, Xujie Song, Wenxuan Wang, Liming Xiao, Jiang Wu, Jingliang Duan, et al. Diffusion actor-critic with entropy regulator. *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

Yixian Zhang, Shu'ang Yu, Tonghe Zhang, Mo Guang, Haojia Hui, Kaiwen Long, Yu Wang, Chao Yu, and Wenbo Ding. SAC flow: Sample-efficient reinforcement learning of flow-based policies via velocity-reparameterized sequential modeling. *International Conference on Learning Representations (ICLR)*, 2026.

Lei Lv, Yunfei Li, Yu Luo, Fuchun Sun, and Xiao Ma. FLAC: Maximum entropy RL via kinetic energy regularized bridge matching. *arXiv preprint arXiv:2602.12829*, 2026.

Zhaoyu Zhu, Shuhan Zhang, Rui Gao, and Shuang Li. Wasserstein proximal policy gradient. *arXiv preprint arXiv:2603.02576*, 2026.

Thanh Xuan Nguyen and Chang D Yoo. One-step flow Q-learning: Addressing the diffusion policy bottleneck in offline reinforcement learning. *International Conference on Learning Representations (ICLR)*, 2026.

Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. *International Conference on Learning Representations (ICLR)*, 2025.

Marc G Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. *International Conference on Machine Learning (ICML)*, 2017.

Aditya Bhatt, Daniel Palenicek, Boris Belousov, Max Argus, Artemij Amiranashvili, Thomas Brox, and Jan Peters. CrossQ: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity. *International Conference on Learning Representations (ICLR)*, 2024.

Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.

Saran Tunyasuvunakool, Alistair Muldal, Yotam Doron, Siqi Liu, Steven Bohez, Josh Merel, Tom Erez, Timothy Lillicrap, Nicolas Heess, and Yuval Tassa. dm_control: Software and tasks for continuous control. *Software Impacts*, 6:100022, 2020.

Carmelo Sferrazza, Dun-Ming Huang, Xingyu Lin, Youngwoon Lee, and Pieter Abbeel. Humanoidbench: Simulated humanoid benchmark for whole-body locomotion and manipulation. *arXiv preprint arXiv:2403.10506*, 2024.

Xiaoyi Dong, Jian Cheng, and Xi Sheryl Zhang. Maximum entropy reinforcement learning with diffusion policy. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 13963–13983, 2025.

Haitong Ma, Tianyi Chen, Kai Wang, Na Li, and Bo Dai. Efficient online reinforcement learning for diffusion policy. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 41837–41853, 2025.

Raj Ghugare and Benjamin Eysenbach. Normalizing flows are capable models for continuous control. In *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2025.

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2021.

Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL `http://jmlr.org/papers/v22/20-1364.html`.

Safa Messaoud, Billel Mokeddem, Zhenghai Xue, Linsey Pang, Bo An, Haipeng Chen, and Sanjay Chawla. S^2AC: Energy-based reinforcement learning with stein soft actor critic. In *International Conference on Learning Representations*, volume 2024, pages 22323–22336, 2024.

Andreas Krause and Volkmar Liebscher. Multimodal projection pursuit using the dip statistic. *Ernst-Moritz-Arndt-Univ., Inst. für Mathematik und Informatik*, 2005.

Prodromos Kolyvakis and Aristidis Likas. A multivariate unimodality test harnessing the dip statistic of mahalanobis distances over random projections. In *Proceedings of the Forty-first Conference on Uncertainty in Artificial Intelligence*, volume 286 of *Proceedings of Machine Learning Research*, pages 2255–2268, 2025.

Daniel Palenicek, Florian Vogt, Joe Watson, Ingmar Posner, and Jan Peters. Xqc: Well-conditioned optimization accelerates deep reinforcement learning. In *International Conference on Learning Representations*, volume 2026, pages 148719–148748, 2026.

Contents

1 Introduction **1**

2 Preliminaries **3**

3 Maximum-Entropy Reinforcement Learning in Path Space **4**

3.1 Path-Space Objective 4

3.2 Unrestricted Soft-Optimal Bridge 4

3.3 Fixed-Base Bridge Choice 5

4 Path-Regularized Generative Actor-Critic **6**

4.1 Soft Bridge Policies 6

4.2 Off-Policy Actor-Critic Training 7

5 Experiments **8**

6 Related Work and Discussion **9**

7 Conclusion and Limitations **10**

A Theorem Proofs **16**

A.1 Proof of Proposition 1 16

A.2 Proof of Theorem 1 16

A.3 Proof of Theorem 2 17

A.4 Proof of Corollary 1 17

A.5 Proof of Lemma 1 18

A.6 Proof of Proposition 2 19

B Algorithm Pseudocode **20**

C 2D Bridge Visualization **20**

D Finite-Step Reference Endpoint Bias **21**

E Detailed Experimental Setup **23**

E.1 Environment and Task Domains 23

E.2 Training Hardware 24

E.3 Implementation Details 24

E.4 Choosing the Control-Energy Budget 25

E.5 Hyperparameters 25

F Extended Experimental Results and Ablations **26**

F.1 Benchmark Results and Sensitivity 26

F.2 Conditional Action Multimodality 29

F.3 Path Regularizer and Base Distribution 30
F.4 Diffusion and Normalizing-Flow Baselines 30
F.5 Actor Cost and Training Wall-Clock Time 31
F.6 Transfer to an XQC Critic Backend 32
F.7 Learned Variance and Fixed-Budget Bridge Depth 32

A Theorem Proofs

A.1 Proof of Proposition 1

Restatement. Let $a = T(z_K)$ be the terminal action. If

$$P_\theta(d\tau \mid s) = \pi_\theta(da \mid s)\, P_\theta(d\tau \mid s, a), \qquad R(d\tau) = u_R(da)\, R(d\tau \mid a),$$

then

$$D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s) \,\|\, R\right) = D_{\mathrm{KL}}\left(\pi_\theta(\cdot \mid s) \,\|\, u_R\right) + \mathbb{E}_{a\sim\pi_\theta}\left[D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s, a) \,\|\, R(\cdot \mid a)\right)\right].$$

Proof. By the stated disintegrations and the terminal map $a = T(z_K)$, the Radon-Nikodym derivative factorizes into an endpoint term and a conditional path term:

$$\log \frac{dP_\theta(\tau \mid s)}{dR(\tau)} = \log \frac{d\pi_\theta(a \mid s)}{du_R(a)} + \log \frac{dP_\theta(\tau \mid s, a)}{dR(\tau \mid a)}. \tag{32}$$

This is the chain rule for relative entropy written at the level of path laws. Taking expectation under $P_\theta(\cdot \mid s)$ and then disintegrating the expectation by $a \sim \pi_\theta(\cdot \mid s)$ gives

$$\begin{aligned} D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s) \,\|\, R\right) = \mathbb{E}_{a\sim\pi_\theta}&\left[\log \frac{d\pi_\theta(a \mid s)}{du_R(a)}\right] \\ &+ \mathbb{E}_{a\sim\pi_\theta}\mathbb{E}_{\tau\sim P_\theta(\cdot\mid s,a)}\left[\log \frac{dP_\theta(\tau \mid s, a)}{dR(\tau \mid a)}\right]. \end{aligned} \tag{33}$$

The first term is $D_{\mathrm{KL}}(\pi_\theta(\cdot \mid s) \,\|\, u_R)$. For each terminal action a, the inner expectation in the second term is $D_{\mathrm{KL}}(P_\theta(\cdot \mid s, a) \,\|\, R(\cdot \mid a))$. This proves Eq. (7). □

A.2 Proof of Theorem 1

Restatement. For fixed s, Q and $\alpha > 0$, optimize Eq. (5) over all path laws $P \ll R$. If

$$Z_Q(s) = \mathbb{E}_{\tau\sim R}\left[\exp\left(Q(s, T(z_K))/\alpha\right)\right] < \infty,$$

then the unique optimum is

$$\frac{dP_Q^\star}{dR}(\tau \mid s) = \frac{\exp\left(Q(s, T(z_K))/\alpha\right)}{Z_Q(s)}.$$

Its terminal action law is

$$\pi_Q^\star(a \mid s) = \frac{u_R(a)\exp\left(Q(s, a)/\alpha\right)}{\int_{\mathcal{A}} u_R(b)\exp\left(Q(s, b)/\alpha\right) db}.$$

Proof. Let $f_s(\tau) = Q(s, T(z_K))$. Since $Z_Q(s) < \infty$, the density in Eq. (9) integrates to one under R, so it defines a valid path law $P_Q^\star$ with $P_Q^\star \ll R$. For any admissible path law $P \ll R$,

$$D_{\mathrm{KL}}\left(P \,\|\, P_Q^\star\right) = \mathbb{E}_{\tau\sim P}\left[\log \frac{dP}{dR}(\tau) - \frac{f_s(\tau)}{\alpha} + \log Z_Q(s)\right]. \tag{34}$$

Rearranging yields the Gibbs variational identity

$$\mathbb{E}_{\tau\sim P}[f_s(\tau)] - \alpha D_{\mathrm{KL}}\left(P \,\|\, R\right) = \alpha \log Z_Q(s) - \alpha D_{\mathrm{KL}}\left(P \,\|\, P_Q^\star\right). \tag{35}$$

The left-hand side is exactly the unrestricted path objective evaluated at P. The right-hand side is upper bounded by $\alpha \log Z_Q(s)$ because relative entropy is nonnegative. The upper bound is attained if and only if $D_{\mathrm{KL}}(P \,\|\, P_Q^\star) = 0$, which holds if and only if $P = P_Q^\star$ almost surely. This proves both optimality and uniqueness.

To compute the terminal marginal, disintegrate the reference as $R(d\tau) = u_R(da)R(d\tau \mid a)$. Since $f_s(\tau) = Q(s, a)$ depends only on the terminal action,

$$P_Q^\star(da \mid s) = \frac{\exp(Q(s, a)/\alpha)}{Z_Q(s)} u_R(da). \tag{36}$$

The normalizer can be written as

$$Z_Q(s) = \int_{\mathcal{A}} u_R(b)\exp(Q(s, b)/\alpha) db, \tag{37}$$

because $\int R(d\tau \mid b) = 1$ for every terminal action b. Substituting this expression gives Eq. (10). □

A.3 Proof of Theorem 2

Restatement. Let

$$Z_Q(s, z_0) = \mathbb{E}_{\tau \sim R(\cdot|z_0)} \left[\exp \left(Q(s, T(z_K))/\alpha \right) \right], \qquad \bar{Z}_Q(s) = \mathbb{E}_{z_0 \sim r_0} \left[Z_Q(s, z_0) \right].$$

Among all path laws with initial marginal p_0, the unique optimum is

$$P^\star_{Q,p_0}(d\tau \mid s) = p_0(dz_0) \, \frac{\exp \left(Q(s, T(z_K))/\alpha \right)}{Z_Q(s, z_0)} R(d\tau_{1:K} \mid z_0).$$

Its objective value is

$$\alpha \mathbb{E}_{z_0 \sim p_0} \left[\log Z_Q(s, z_0) \right] - \alpha D_{\mathrm{KL}} \left(p_0 \,\|\, r_0 \right).$$

Proof. Any feasible fixed-base law can be written as

$$P(d\tau \mid s) = p_0(dz_0) P(d\tau_{1:K} \mid z_0, s). \tag{38}$$

The reference has the corresponding decomposition

$$R(d\tau) = r_0(dz_0) R(d\tau_{1:K} \mid z_0). \tag{39}$$

The likelihood ratio therefore separates into an initial-base term and a conditional path term:

$$\log \frac{dP}{dR}(\tau \mid s) = \log \frac{dp_0}{dr_0}(z_0) + \log \frac{dP(\cdot \mid z_0, s)}{dR(\cdot \mid z_0)}(\tau_{1:K}). \tag{40}$$

Taking expectation under P gives

$$D_{\mathrm{KL}} \left(P \,\|\, R \right) = D_{\mathrm{KL}} \left(p_0 \,\|\, r_0 \right) + \mathbb{E}_{z_0 \sim p_0} \left[D_{\mathrm{KL}} \left(P(\cdot \mid z_0, s) \,\|\, R(\cdot \mid z_0) \right) \right]. \tag{41}$$

The value term also decomposes conditionally:

$$\mathbb{E}_{\tau \sim P} \left[Q(s, T(z_K)) \right] = \mathbb{E}_{z_0 \sim p_0} \mathbb{E}_{\tau_{1:K} \sim P(\cdot|z_0,s)} \left[Q(s, T(z_K)) \right]. \tag{42}$$

Hence the fixed-base objective is an average, over $z_0 \sim p_0$, of independent conditional variational problems minus the initial KL constant. For each fixed z_0, define

$$f_{s,z_0}(\tau_{1:K}) = Q(s, T(z_K)). \tag{43}$$

Applying the Gibbs variational identity to the conditional reference $R(\cdot \mid z_0)$ gives

$$\begin{aligned} \Psi_{z_0}(P) &:= \mathbb{E}_{\tau_{1:K} \sim P(\cdot|z_0,s)} \left[f_{s,z_0}(\tau_{1:K}) \right] - \alpha D_{\mathrm{KL}} \left(P(\cdot \mid z_0, s) \,\|\, R(\cdot \mid z_0) \right) \\ &= \alpha \log Z_Q(s, z_0) - \alpha D_{\mathrm{KL}} \left(P(\cdot \mid z_0, s) \,\|\, P^\star(\cdot \mid z_0, s) \right), \end{aligned} \tag{44}$$

where the unique conditional optimizer satisfies

$$\frac{dP^\star(\cdot \mid z_0, s)}{dR(\cdot \mid z_0)}(\tau_{1:K}) = \frac{\exp(Q(s, T(z_K))/\alpha)}{Z_Q(s, z_0)}, \tag{45}$$

and where $Z_Q(s, z_0)$ is the conditional partition function in Theorem 2. The conditional KL term is nonnegative and vanishes only at this optimizer, so the optimizer is unique for p_0-almost every z_0. Averaging the conditional value $\alpha \log Z_Q(s, z_0)$ under p_0 and subtracting $\alpha D_{\mathrm{KL}}(p_0 \,\|\, r_0)$ gives Eq. (14). Substituting the conditional optimizer into the fixed-base disintegration gives Eq. (13). □

A.4 Proof of Corollary 1

Restatement. Under the assumptions of Theorem 2, the loss relative to the unrestricted optimum is

$$\Delta(s) = \alpha \left(\log \bar{Z}_Q(s) - \mathbb{E}_{z_0 \sim p_0} \left[\log Z_Q(s, z_0) \right] + D_{\mathrm{KL}} \left(p_0 \,\|\, r_0 \right) \right) \geq 0.$$

Equality holds if and only if p_0 equals the initial marginal of the unrestricted tilted bridge. If $p_0 = r_0$, this is equivalent to $Z_Q(s, z_0)$ being constant for p_0-almost every z_0. Moreover,

$$D_{\mathrm{KL}} \left(P^\star_{Q,p_0}(\cdot \mid s) \,\|\, P^\star_Q(\cdot \mid s) \right) = \frac{\Delta(s)}{\alpha},$$

and

$$D_{\mathrm{KL}} \left(\pi^\star_{Q,p_0}(\cdot \mid s) \,\|\, \pi^\star_Q(\cdot \mid s) \right) \leq \frac{\Delta(s)}{\alpha}.$$

Proof. The unrestricted partition function can be decomposed by the reference base law:

$$\begin{aligned} Z_Q(s) &= \mathbb{E}_{\tau\sim R}\left[\exp(Q(s,T(z_K))/\alpha)\right] \\ &= \mathbb{E}_{z_0\sim r_0}\left[Z_Q(s,z_0)\right] \\ &= \bar{Z}_Q(s). \end{aligned} \tag{46}$$

Therefore Theorem 1 gives unrestricted value $\alpha \log \bar{Z}_Q(s)$, while Theorem 2 gives fixed-base value

$$\alpha\mathbb{E}_{z_0\sim p_0}\left[\log Z_Q(s,z_0)\right] - \alpha D_{\mathrm{KL}}\left(p_0 \,\|\, r_0\right). \tag{47}$$

Their difference is Eq. (15). This difference is nonnegative because it is the initial-law KL to the unrestricted tilted bridge. Indeed, the unrestricted tilted bridge has initial marginal

$$P^\star_{Q,0}(dz_0 \mid s) = \frac{Z_Q(s,z_0)}{\bar{Z}_Q(s)} r_0(dz_0). \tag{48}$$

Thus

$$\begin{aligned} D_{\mathrm{KL}}\left(p_0 \,\|\, P^\star_{Q,0}(\cdot \mid s)\right) &= \mathbb{E}_{z_0\sim p_0}\left[\log \frac{dp_0}{dr_0}(z_0) + \log \bar{Z}_Q(s) - \log Z_Q(s,z_0)\right] \\ &= \frac{\Delta(s)}{\alpha}. \end{aligned} \tag{49}$$

This proves nonnegativity and the equality condition. If $p_0 = r_0$, the condition reduces to $Z_Q(s,z_0)$ being constant under p_0.

It remains to relate this gap to the path and endpoint laws. The unrestricted tilted bridge satisfies

$$\frac{dP^\star_Q}{dR}(\tau \mid s) = \frac{\exp(Q(s,T(z_K))/\alpha)}{\bar{Z}_Q(s)}, \tag{50}$$

while the fixed-base optimum satisfies

$$\frac{dP^\star_{Q,p_0}}{dR}(\tau \mid s) = \frac{dp_0}{dr_0}(z_0)\frac{\exp(Q(s,T(z_K))/\alpha)}{Z_Q(s,z_0)}. \tag{51}$$

Therefore

$$\begin{aligned} D_{\mathrm{KL}}\left(P^\star_{Q,p_0} \,\|\, P^\star_Q\right) &= \mathbb{E}_{z_0\sim p_0}\left[\log \frac{dp_0}{dr_0}(z_0) + \log \frac{\bar{Z}_Q(s)}{Z_Q(s,z_0)}\right] \\ &= \frac{\Delta(s)}{\alpha}. \end{aligned} \tag{52}$$

The expectation above is under $P^\star_{Q,p_0}$. Its initial marginal is still p_0 by construction, which is why the expectation reduces to $z_0 \sim p_0$. Finally, the endpoint action is the measurable map $\tau \mapsto T(z_K)$. Data processing for relative entropy under this map gives

$$D_{\mathrm{KL}}\left(\pi^\star_{Q,p_0}(\cdot \mid s) \,\|\, \pi^\star_Q(\cdot \mid s)\right) \le D_{\mathrm{KL}}\left(P^\star_{Q,p_0}(\cdot \mid s) \,\|\, P^\star_Q(\cdot \mid s)\right), \tag{53}$$

which proves Eq. (17). □

A.5 Proof of Lemma 1

Restatement. Let

$$P_\theta(d\tau \mid s) = p_0(dz_0)\prod_{k=0}^{K-1} q_{\theta,k}(dz_{k+1} \mid z_k, s), \qquad R(d\tau) = r_0(dz_0)\prod_{k=0}^{K-1} r_k(dz_{k+1} \mid z_k).$$

Then

$$D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s) \,\|\, R\right) = D_{\mathrm{KL}}\left(p_0 \,\|\, r_0\right) + \sum_{k=0}^{K-1} \mathbb{E}_{\tau\sim P_\theta}\left[D_{\mathrm{KL}}\left(q_{\theta,k}(\cdot \mid z_k, s) \,\|\, r_k(\cdot \mid z_k)\right)\right].$$

For Gaussian local kernels, if $\mathcal{C}_\theta(s,\tau)$ is the sum of the local Gaussian KL terms, then

$$D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s) \,\|\, R\right) = D_{\mathrm{KL}}\left(p_0 \,\|\, r_0\right) + \mathbb{E}_{\tau\sim P_\theta(\cdot|s)}\left[\mathcal{C}_\theta(s,\tau)\right].$$

Proof. The path likelihood ratio separates into an initial-base term and local transition terms:

$$\log \frac{dP_\theta(\tau \mid s)}{dR(\tau)} = \log \frac{dp_0}{dr_0}(z_0) + \sum_{k=0}^{K-1} \log \frac{q_{\theta,k}(z_{k+1} \mid z_k, s)}{r_k(z_{k+1} \mid z_k)}. \tag{54}$$

Taking expectation under the actor path law gives the initial contribution

$$\mathbb{E}_{z_0 \sim p_0}\left[\log \frac{dp_0}{dr_0}(z_0)\right] = D_{\mathrm{KL}}\left(p_0 \,\|\, r_0\right). \tag{55}$$

$$\begin{aligned}\mathbb{E}_{\tau \sim P_\theta(\cdot|s)}\left[\log \frac{q_{\theta,k}(z_{k+1} \mid z_k, s)}{r_k(z_{k+1} \mid z_k)}\right] &= \mathbb{E}_{z_k \sim P_{\theta,k}(\cdot|s)} \mathbb{E}_{z_{k+1} \sim q_{\theta,k}(\cdot|z_k,s)}\left[\log \frac{q_{\theta,k}(z_{k+1} \mid z_k, s)}{r_k(z_{k+1} \mid z_k)}\right] \\ &= \mathbb{E}_{z_k \sim P_{\theta,k}(\cdot|s)}\left[D_{\mathrm{KL}}\left(q_{\theta,k}(\cdot \mid z_k, s) \,\|\, r_k(\cdot \mid z_k)\right)\right], \end{aligned} \tag{56}$$

where $P_{\theta,k}(\cdot \mid s)$ is the actor marginal law of z_k. Summing this identity over k gives Eq. (22). If the local kernels are Gaussian, each local KL has a closed form. Defining $\mathcal{C}_\theta(s,\tau)$ as the sum of those local Gaussian KLs gives Eq. (23). □

A.6 Proof of Proposition 2

Restatement. Let $P_Q^\star$ be the unrestricted soft-optimal bridge in Theorem 1. For any finite-step bridge actor with fixed base law $p_0 \ll r_0$ satisfying Lemma 1,

$$\mathcal{L}_{\mathrm{actor}}(\theta \mid s) = \mathbb{E}_{\tau \sim P_\theta(\cdot|s)}\left[\alpha\, \mathcal{C}_\theta(s,\tau) - Q(s, T(z_K))\right]$$

satisfies

$$\mathcal{L}_{\mathrm{actor}}(\theta \mid s) = \alpha D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s) \,\|\, P_Q^\star(\cdot \mid s)\right) - \alpha \log Z_Q(s) - \alpha D_{\mathrm{KL}}\left(p_0 \,\|\, r_0\right).$$

Consequently, minimizing this loss over the unrestricted fixed-base path-law family has global minimizer $P^\star_{Q,p_0}$.

Proof. Fix a state s. By Lemma 1, the sampled control energy satisfies

$$\mathbb{E}_{\tau \sim P_\theta(\cdot|s)}\left[\mathcal{C}_\theta(s,\tau)\right] = D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s) \,\|\, R\right) - D_{\mathrm{KL}}\left(p_0 \,\|\, r_0\right). \tag{57}$$

The unrestricted soft-optimal bridge from Theorem 1 has likelihood ratio

$$\log \frac{dP_Q^\star}{dR}(\tau \mid s) = \frac{Q(s, T(z_K))}{\alpha} - \log Z_Q(s). \tag{58}$$

Expanding the reverse KL to this ideal bridge gives

$$\begin{aligned} &\alpha D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s) \,\|\, P_Q^\star(\cdot \mid s)\right) \\ &= \alpha \mathbb{E}_{\tau \sim P_\theta(\cdot|s)}\left[\log \frac{dP_\theta}{dR}(\tau \mid s) - \frac{Q(s, T(z_K))}{\alpha} + \log Z_Q(s)\right] \\ &= \alpha D_{\mathrm{KL}}\left(P_\theta(\cdot \mid s) \,\|\, R\right) - \mathbb{E}_{\tau \sim P_\theta}\left[Q(s, T(z_K))\right] + \alpha \log Z_Q(s) \\ &= \mathcal{L}_{\mathrm{actor}}(\theta \mid s) + \alpha \log Z_Q(s) + \alpha D_{\mathrm{KL}}\left(p_0 \,\|\, r_0\right). \end{aligned} \tag{59}$$

Rearranging gives Eq. (30). Since the projection is optimized over the unrestricted fixed-base family, Theorem 2 identifies the global minimizer as $P^\star_{Q,p_0}$. The finite neural Gaussian Markov actor used by the algorithm is a restricted parameterization of this family, so the proof gives the population projection target rather than a global optimization guarantee for SGD. □

B Algorithm Pseudocode

Algorithm 1 `SoftGAC` off-policy training

Require: replay buffer $\mathcal{D}$, actor P_θ, target actor $P_{\bar{\theta}}$, critic Q_ϕ, target critic $Q_{\bar{\phi}}$, temperature α, target cost $\mathcal{C}_{\text{target}}$, discount γ, Polyak coefficient ρ

initialize $\mathcal{D}$, θ, ϕ, $\bar{\theta} \leftarrow \theta$, $\bar{\phi} \leftarrow \phi$
for each environment step **do**
  sample a bridge path $\tau \sim P_\theta(\cdot \mid s)$ and execute $a = T(z_K)$
  store (s, a, r, s', d) in $\mathcal{D}$
  for each gradient update **do**
    sample a replay batch $(s, a, r, s', d) \sim \mathcal{D}$
    sample next paths $\tau' \sim P_{\bar{\theta}}(\cdot \mid s')$
    compute next actions $a' = T(z'_K)$ and bridge costs $\mathcal{C}_{\bar{\theta}}(s', \tau')$
    build soft targets $y = r + \gamma(1-d)\left[Q^{\min}_{\bar{\phi}}(s', a') - \alpha \mathcal{C}_{\bar{\theta}}(s', \tau')\right]$
    update Q_ϕ toward y with the chosen off-policy critic loss
    if the policy-delay interval is reached **then**
      sample current paths $\tau \sim P_\theta(\cdot \mid s)$
      compute current actions $\tilde{a} = T(z_K)$ and bridge costs $\mathcal{C}_\theta(s, \tau)$
      update θ to minimize $\mathbb{E}[\alpha \mathcal{C}_\theta(s, \tau) - Q^{\min}_\phi(s, \tilde{a})]$
      update $\log \alpha$ to minimize $\mathbb{E}[\alpha(\mathcal{C}_{\text{target}} - \mathcal{C}_\theta(s, \tau))]$ with $\alpha = \exp(\log \alpha)$
    end if
    if using target networks and the target-update interval is reached **then**
      soft-update target networks $\bar{\theta} \leftarrow \rho\bar{\theta} + (1-\rho)\theta$ and $\bar{\phi} \leftarrow \rho\bar{\phi} + (1-\rho)\phi$
    end if
  end for
end for

C 2D Bridge Visualization

Figure 5 visualizes the bridge mechanism in a controlled two-dimensional fixed-critic setting. This diagnostic setting isolates the actor update by training a bridge policy against a hand-designed multimodal value function. It makes the finite-step path distribution visible as it moves from the base law to the terminal action law under different control-energy budgets.

The bounded action is $a = (a_1, a_2) \in (-1, 1)^2$. We define an unnormalized fixed critic as a log-sum-exp mixture of three Gaussian-like modes,

$$Q_{\text{raw}}(a) = \log \sum_{m=1}^{3} w_m \exp\left(-\frac{1}{2}\left\|\frac{a - c_m}{\sigma_m}\right\|_2^2\right), \tag{60}$$

with centers

$$c_1 = (-0.68, 0.50), \qquad c_2 = (0.62, 0.50), \qquad c_3 = (-0.06, -0.58), \tag{61}$$

axis-wise widths

$$\sigma_1 = (0.24, 0.24), \qquad \sigma_2 = (0.24, 0.24), \qquad \sigma_3 = (0.34, 0.32), \tag{62}$$

and weights $(w_1, w_2, w_3) = (0.90, 1.00, 0.70)$. We normalize the critic to $[0, 1]$ over a dense action grid and use this normalized value as $Q(s, a)$ for a single dummy state s. The latent landscape shown in the second column is the pullback $Q(s, \tanh z)$.

The reference process is the same finite-step reference used in the method section. We sample the base action from an approximately uniform law on $[-0.995, 0.995]^2$ and set $z_0 = \operatorname{arctanh}(a_0)$. The reference latent process uses $K = 6$ Euler steps with $h = 1/K$,

$$z_{k+1} = z_k - 2h \tanh(z_k) + \sqrt{2h}\,\epsilon_k, \qquad \epsilon_k \sim \mathcal{N}(0, I). \tag{63}$$

This finite chain approximates the continuous reference whose stationary action law after $\tanh$ is uniform. We intentionally show the finite-step reference rather than the continuous-limit uniform distribution because the algorithm optimizes the finite-step path KL.

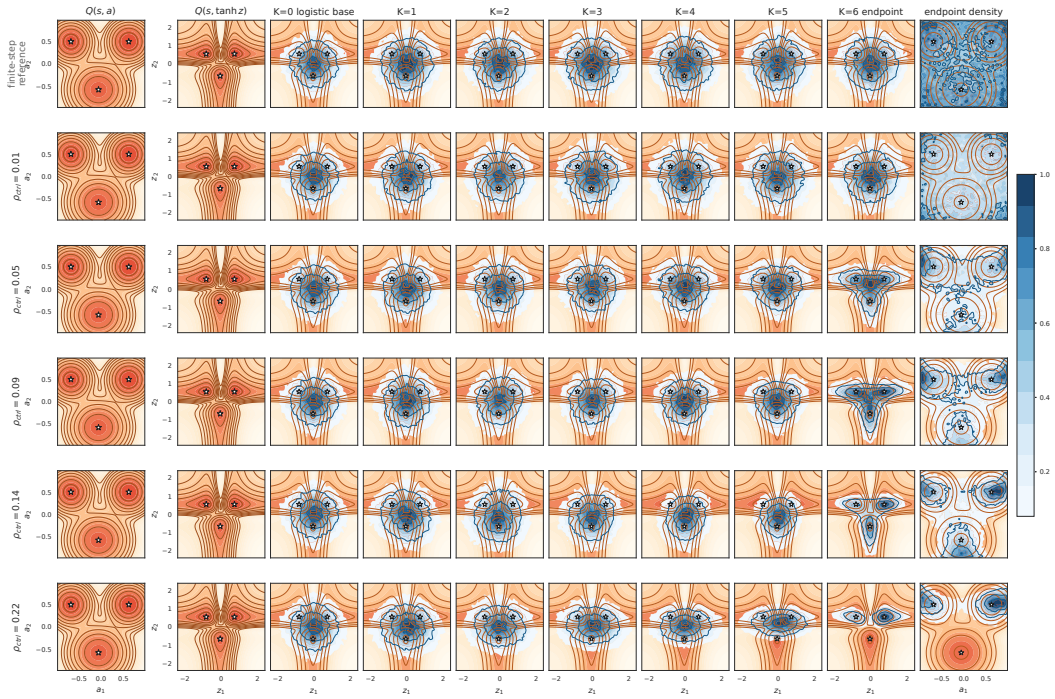


Figure 5: 2D bridge visualization. Each row corresponds to a different control-energy budget. Columns show the action-space critic, the corresponding pre-tanh latent critic, intermediate latent bridge densities and the final action-space policy density.

For each control-energy budget, we train a separate bridge actor with $K = 6$ action-dimensional latent transitions. Given a sampled logistic reference base latent and Gaussian transition noise, the actor produces a path $\tau = (z_0, \ldots, z_K)$ and terminal action $a = \tanh(z_K)$. We optimize the sampled actor objective

$$\mathbb{E}_{\tau \sim P_\theta} \left[\alpha \, \mathcal{C}_\theta(s, \tau) - Q(s, \tanh z_K) \right], \tag{64}$$

where $\mathcal{C}_\theta$ is the finite-step local Gaussian control energy from Lemma 1. The temperature is tuned with the same budget form as the main algorithm,

$$\mathcal{C}_{\text{target}} = \rho_{\text{ctrl}} K d_a, \qquad d_a = 2. \tag{65}$$

The rows in Figure 5 use

$$\rho_{\text{ctrl}} \in \{0.01, 0.05, 0.09, 0.14, 0.22\}. \tag{66}$$

Small budgets keep the actor close to the reference bridge and preserve broad coverage. Larger budgets allow stronger value guidance and concentrate the endpoint density near higher-value modes. The first two columns show $Q(s, a)$ in action space and $Q(s, \tanh z)$ in pre-tanh latent space. The next columns show latent densities at bridge steps $K = 0, \ldots, 6$. The final column maps terminal latents back to action space and overlays the resulting policy density on the action-space critic.

D Finite-Step Reference Endpoint Bias

In this paper, we use a finite-step reference bridge to approximate the ideal continuous reference whose endpoint action law is uniform. This approximation affects only the endpoint marginal law used as the reference prior. It is distinct from the path-wise KL used by the actor update, which remains exact for the chosen finite-step actor-reference pair. We quantify this endpoint effect in Figure 6.

Let $q_{\text{ref}}(z) = \frac{1}{2}\text{sech}^2(z)$ be the one-dimensional pre-tanh density whose image under $\tanh$ is uniform. For a reference bridge with K Euler steps, let p_K be the one-dimensional terminal latent density produced by

$$z_{k+1} = z_k - 2h \tanh(z_k) + \sqrt{2h}\, \epsilon_k, \qquad h = \frac{1}{K}. \tag{67}$$

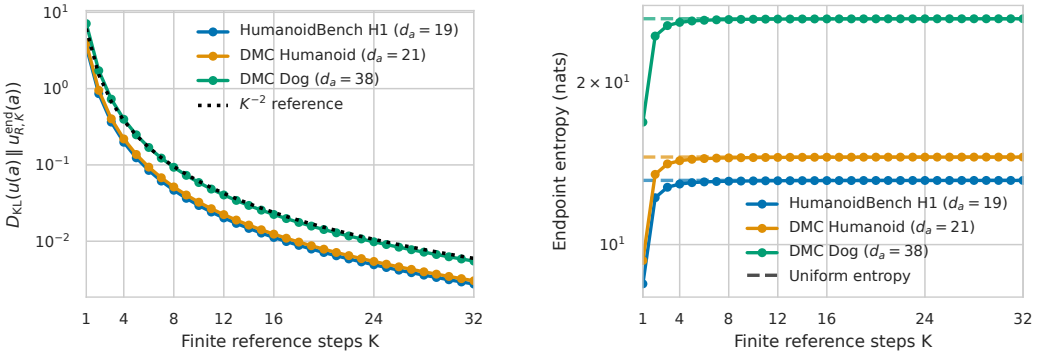


Figure 6: Finite-step reference endpoint bias. Left shows the endpoint action-marginal KL from the ideal uniform prior to the finite-step reference endpoint law. Right shows the finite-step endpoint entropy and the corresponding uniform entropy baselines. The three curves use the action dimensions of HumanoidBench H1, DMC Humanoid and DMC Dog.

The map $\tanh$ is bijective between latent space and the normalized action interval, so endpoint KL values can be computed in latent space without estimating an action-space density. For example,

$$D_{\mathrm{KL}}\big(u(a) \,\big\|\, u^{\mathrm{end}}_{R,K}(a)\big) = D_{\mathrm{KL}}(q_{\mathrm{ref}}(z) \,\|\, p_K(z))\,. \tag{68}$$

The implemented reference factorizes across action dimensions, so the d_a-dimensional laws are product measures and relative entropy is additive across coordinates. It is therefore enough to compute the one-dimensional terminal density and multiply the resulting KL by d_a. Let $\mathcal{T}_h$ be the one-dimensional Euler transition operator,

$$(\mathcal{T}_h f)(z') = \int \mathcal{N}(z' \,|\, z - 2h\tanh z, 2h)\, f(z)\, dz, \qquad h = \frac{1}{K}. \tag{69}$$

Then $p_K = \mathcal{T}_h^K q_{\mathrm{ref}}$, and the d_a-dimensional endpoint-prior gap has the exact finite-K form

$$G_K(d_a) = D_{\mathrm{KL}}\big(u(a) \,\big\|\, u^{\mathrm{end}}_{R,K}(a)\big) = d_a \int q_{\mathrm{ref}}(z) \log \frac{q_{\mathrm{ref}}(z)}{p_K(z)} dz. \tag{70}$$

Thus the dependence on d_a is exactly linear. The dependence on K comes only from the Euler discretization error in p_K. Under the standard first-order density expansion for Euler discretization over a fixed time horizon, $p_K(z) = q_{\mathrm{ref}}(z)(1 + K^{-1}r(z) + O(K^{-2}))$ with $\int q_{\mathrm{ref}}(z) r(z)\, dz = 0$. The first-order term cancels in the KL because of normalization, so the leading contribution is quadratic in the density error. Substituting this expansion into Eq. (70) gives

$$G_K(d_a) = \frac{d_a}{2K^2} \int q_{\mathrm{ref}}(z) r(z)^2\, dz + O\left(\frac{d_a}{K^3}\right). \tag{71}$$

Thus the leading endpoint-prior gap scales as $O(d_a/K^2)$ whenever this expansion holds. This rate explains the observed finite-step bias but is not an assumption used by the algorithm. The entropy deficit has the same leading rate because it equals $D_{\mathrm{KL}}(u^{\mathrm{end}}_{R,K} \| u)$. Likewise, the endpoint entropy satisfies

$$H\big(u^{\mathrm{end}}_{R,K}\big) = d_a \log 2 - D_{\mathrm{KL}}\big(u^{\mathrm{end}}_{R,K} \,\big\|\, u\big)\,, \tag{72}$$

where $d_a \log 2$ is the d_a-dimensional uniform endpoint entropy. We compute the one-dimensional density deterministically on a dense grid by applying the Gaussian transition kernel, then report the corresponding d_a-dimensional quantities for the action dimensions used in our experiments.

The bias decreases quickly with K. At the default $K = 6$, the endpoint KL $D_{\mathrm{KL}}(u \,\|\, u^{\mathrm{end}}_{R,K})$ is approximately 0.085, 0.094 and 0.169 nats for $d_a = 19$, 21 and 38. The endpoint entropy is already close to the uniform entropy in all three domains, while the network is still shallow to optimize. This supports the interpretation that the finite reference mainly provides an implementable high-entropy prior, while the algorithm optimizes the exact finite-step path regularizer defined by that prior.

E Detailed Experimental Setup

E.1 Environment and Task Domains

Our benchmark contains 12 continuous-control tasks from three domains. Figure 7 shows representative rendered states. All experiments use vector observations and continuous bounded actions. The reported dimensions are the observation and action dimensions after the environment wrappers used by our training code.

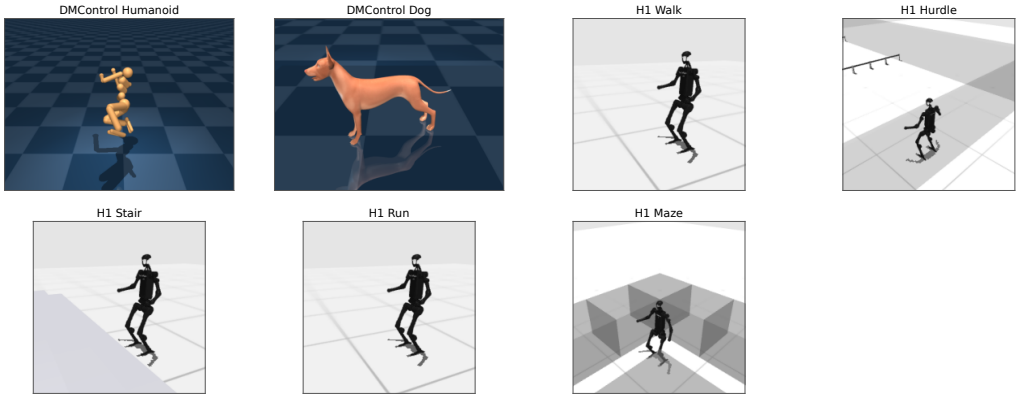


Figure 7: Representative screenshots of the benchmark domains and H1 tasks.

Table 1: Benchmark task dimensions.

Domain	Task	Observation dim.	Action dim.
DMC Humanoid	`dm_control/humanoid-run`	67	21
DMC Humanoid	`dm_control/humanoid-walk`	67	21
DMC Humanoid	`dm_control/humanoid-stand`	67	21
DMC Dog	`dm_control/dog-run`	223	38
DMC Dog	`dm_control/dog-trot`	223	38
DMC Dog	`dm_control/dog-walk`	223	38
DMC Dog	`dm_control/dog-stand`	223	38
HumanoidBench H1	`h1-walk-v0`	51	19
HumanoidBench H1	`h1-hurdle-v0`	51	19
HumanoidBench H1	`h1-stair-v0`	51	19
HumanoidBench H1	`h1-run-v0`	51	19
HumanoidBench H1	`h1-maze-v0`	51	19

DMC Humanoid evaluates high-dimensional biped locomotion with running, walking and standing objectives. DMC Dog evaluates quadruped control with a larger observation and action space, including fast running, trotting, walking and standing. HumanoidBench H1 uses a humanoid robot with 19 actuated degrees of freedom. The walk and run tasks measure whole-body locomotion, while hurdle, stair and maze add obstacle negotiation, contact-rich terrain interaction and navigation constraints.

Benchmark assets and licenses. We use the DeepMind Control Suite through `dm_control` version 1.0.39 from `https://github.com/google-deepmind/dm_control`, which is released under the Apache License 2.0. We use HumanoidBench through the official repository's main branch from `https://github.com/carlosferrazza/humanoid-bench`, which is released under the MIT License and includes third-party notices in its repository license file.

Baseline implementation references. All baseline results are produced by our unified JAX codebase. We use the official repositories in Table 2 as implementation references, then re-implement the algorithm-specific actor, target construction and auxiliary losses in JAX inside our training stack.

This gives each method the same logging, replay, evaluation and experiment infrastructure while preserving the algorithm-specific components required by the original implementations.

Table 2: Official implementation repositories used as baseline references.

Method	Official repository
FLAC	`https://github.com/bytedance/FLAC`
DIME	`https://github.com/ALRhub/DIME`
FlowRL	`https://github.com/bytedance/FlowRL`
QSM	`https://github.com/escontra/score_matching_rl`
QVPO	`https://github.com/wadx2019/qvpo`
CrossQ-SAC	`https://github.com/adityab/CrossQ`
DACER	`https://github.com/happy-yan/DACER-Diffusion-with-Online-RL`
MaxEntDP	`https://github.com/diffusionyes/MaxEntDP`
SDAC	`https://github.com/mahaitongdae/diffusion_policy_online_rl`
Normalizing Flow	`https://github.com/Princeton-RL/normalising-flows-4reinforcement-learning`

E.2 Training Hardware

We train agents on Google Cloud TPU v4-16 and v6e-8 VMs, running independent seeds in parallel with one TPU core per seed. On the v6e hosts, Linux reports two AMD EPYC 9B14 sockets with 90 cores per socket, 180 logical CPUs, one thread per core and approximately 1.4 TiB of system memory. The machines run Ubuntu Linux with kernel 6.8 on x86_64 CPUs. Appendix F reports training wall-clock curves, and Appendix F.5 specifies the hardware used for inference and training-time comparisons.

E.3 Implementation Details

The `SoftGAC` actor operates in pre-tanh latent space and samples a tensor of noises with shape $B \times (K+1) \times d_a$. The first slice gives the base latent z_0. By default, we use the logistic base by sampling $a_0 \sim \mathrm{Unif}((-1,1)^{d_a})$ and setting $z_0 = \mathrm{arctanh}(a_0)$. The actor forward pass is:

```
def bridge_actor(obs, noise):
    z = noise[:, 0]                          # base latent z_0
    latents, drifts, sigmas = [z], [], []
    h = 1.0 / K
    for k in range(K):
        x = concat(obs, z)
        x = LayerNorm(x)
        x = Dense(hidden_size)(x)
        x = elu(x)
        x = LayerNorm(x)
        drift = Dense(action_dim)(x)
        sigma = softplus(Dense(action_dim)(x))
        z = z + h * drift + sqrt(2 * h) * sigma * noise[:, k + 1]
        latents.append(z)
        drifts.append(drift)
        sigmas.append(sigma)
    action = action_scale * tanh(z) + action_bias
    return action, latents, drifts, sigmas
```

Each block uses one hidden layer with layer normalization before and after the ELU activation, followed by drift and positive diagonal-scale heads. Main runs use $K = 6$ blocks and width 512, with width 256 for DMC Dog to keep the actor parameter budget close to the baselines. The actor update minimizes $\alpha \mathcal{C}_\theta(s,\tau) - Q_\phi^{\min}(s,a)$. The control energy $\mathcal{C}_\theta$ is the accumulated closed-form Gaussian transition KL to the reference bridge, so no endpoint entropy estimator is used. The temperature $\alpha = \exp(\log \alpha)$ is tuned toward $\rho_{\mathrm{ctrl}} K d_a$, with default $\rho_{\mathrm{ctrl}} = 0.2$. The main critic is the same twin C51 critic with CrossQ-style update adopted in DIME (Celik et al., 2025).

E.4 Choosing the Control-Energy Budget

The target control-energy budget is meant to set how strongly the value term can move the bridge away from the high-entropy reference, not to introduce a task-specific objective. A useful way to choose its scale is to ask how far one action dimension should be allowed to move under value-guided control. The path KL factorizes over K transitions and d_a action dimensions, so we set

$$\mathcal{C}_{\text{target}} = \rho_{\text{ctrl}} K d_a. \tag{73}$$

This makes ρ_{ctrl} the average local KL budget, measured in nats, per transition and per action dimension. Equivalently, each action dimension receives a total path budget of $\rho_{\text{ctrl}} K$ along the full bridge.

This scale has a simple endpoint interpretation. Ignore the learned variance term for a moment and assume that the actor and reference transitions share covariance $2hI$, where $h = 1/K$. If the actor adds a one-dimensional mean control δz_k at step k, the local KL contribution is

$$D_{\text{KL}}\left(\mathcal{N}(\mu_R + \delta z_k, 2h) \,\|\, \mathcal{N}(\mu_R, 2h)\right) = \frac{(\delta z_k)^2}{4h}. \tag{74}$$

For a desired terminal displacement Δz in one pre-tanh action dimension, the cheapest constant-speed control has $\delta z_k \approx \Delta z / K$. The accumulated control energy is then

$$\sum_{k=0}^{K-1} \frac{(\Delta z/K)^2}{4h} = \frac{(\Delta z)^2}{4}. \tag{75}$$

Thus a budget $\rho_{\text{ctrl}} K$ per action dimension roughly permits a terminal pre-tanh displacement

$$|\Delta z| \approx 2\sqrt{\rho_{\text{ctrl}} K}. \tag{76}$$

If we want the actor to be able to push a dimension close to the saturated action region, say $|a| \approx 0.95$ to 0.98, the corresponding pre-tanh scale is $z_{\text{sat}} = \operatorname{arctanh}(|a|) \approx 1.8$ to 2.3. Matching $\rho_{\text{ctrl}} K \approx z_{\text{sat}}^2/4$ gives $\rho_{\text{ctrl}} \approx z_{\text{sat}}^2/(4K)$. For the main bridge depth $K = 6$, this range is about 0.14 to 0.22. We therefore use $\rho_{\text{ctrl}} = 0.2$ as a moderate default. It allows value-guided transport toward near-saturated high-value actions, but still assigns a visible cost to collapse away from the broad reference. Smaller budgets keep broader action coverage, while larger budgets permit sharper value-guided concentration. This is the same qualitative trend visualized in Appendix C.

E.5 Hyperparameters

Table 3 reports the actor and main critic parameter counts by domain, and Table 4 summarizes the main hyperparameters. All methods use 8 seeds and Adam optimizers. To reduce critic-side confounding, we use the same twin C51 main critic and CrossQ-style main critic update adopted in DIME (Celik et al., 2025) across the compared methods. When an algorithm includes additional auxiliary networks, such as FlowRL's buffer critic, we keep the corresponding architecture and update rule from the official implementation.

Table 3: Actor and main critic parameter counts in the experiment domains, in millions of parameters.

	DMC Humanoid		DMC Dog		HumanoidBench	
Algorithm	Actor	Critic	Actor	Critic	Actor	Critic
`SoftGAC`	0.410	9.19	0.526	9.90	0.342	9.11
FLAC	0.322	9.19	0.419	9.90	0.311	9.11
DIME	0.423	9.19	0.472	9.90	0.418	9.11
FlowRL	0.322	9.19	0.419	9.90	0.311	9.11
QSM	0.614	9.19	0.712	9.90	0.604	9.11
QVPO	0.369	9.19	0.466	9.90	0.359	9.11
CrossQ-SAC	0.321	9.19	0.419	9.90	0.311	9.11

Table 4: Main hyperparameters used in the experiments.

Hyperparameter	SoftGAC	FLAC	DIME	FlowRL	QSM	QVPO	CrossQ-SAC
UTD ratio	2	2	2	2	2	2	2
Discount	0.99	0.99	0.99	0.99	0.99	0.99	0.99
Batch size	256	256	256	256	256	256	256
Buffer size	10^6	10^6	10^6	10^6	10^6	10^6	10^6
Learn starts	5000	5000	5000	5000	5000	5000	5000
Actor lr	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}
Critic lr	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}
Policy delay	2	2	2	2	2	2	2
Regularizer	path KL	kinetic	entropy bd.	CFM	score-Q	weighted-Q	entropy
Target entropy/energy	$0.2Kd_a$	$0.5d_a$	$4d_a$	N/A	N/A	N/A	$-d_a$
Temp. lr	10^{-3}	$3 \cdot 10^{-5}$	10^{-3}	N/A	N/A	N/A	10^{-3}
Critic depth	2	2	2	2	2	2	2
Critic hidden size	2048	2048	2048	2048	2048	2048	2048
Num. bins	101	101	101	101	101	101	101
Actor depth	6	2	3	2	2	2	2
Actor width	256/512[†]	512	256	512	512	512	512
Multi-modal	yes	yes	yes	yes	yes	yes	no
Prior dist.	Logistic	clip $\mathcal{N}(0, I)$	$\mathcal{N}(0, 2.5^2 I)$	clip $\mathcal{N}(0, I)$	$\mathcal{N}(0, I)$	$\mathcal{N}(0, I)$	$\mathcal{N}(0, I)$
Iteration	1	2	16	2	5	16	1

[†] SoftGAC uses width 256 only for DMC Dog and width 512 otherwise for controlling the parameter count.

F Extended Experimental Results and Ablations

Unless stated otherwise, return tables report final IQM $\pm$ standard deviation across eight training seeds. We compute the IQM across seeds at each evaluation checkpoint and average the IQMs of the final four checkpoints. The standard deviation is computed across the eight per-seed returns, each averaged over those same checkpoints. Average entries are unweighted means of task-level IQMs.

F.1 Benchmark Results and Sensitivity

Figures 8 and 9 report learning curves and final IQM returns across all 12 benchmark tasks. Figure 10 compares per-task returns with actor inference latency. Figure 11 shows training wall-clock curves on Humanoid Run, Dog Run and H1 Hurdle using the median elapsed time across seeds at each evaluation step. Figures 12, 14 and 13 examine sensitivity to actor width, bridge depth and control budget.

SoftGAC achieves substantial gains on Humanoid Run, Dog Run, H1 Hurdle, H1 Stair and H1 Run, which involve high-dimensional locomotion and long-horizon credit assignment. On easier standing and walking tasks, several algorithms reach similarly high final returns, with larger differences in sample efficiency. The learning curves show when these differences emerge, while the final IQMs summarize performance over the last four evaluations.

The compute-return plots make the Pareto structure explicit. CrossQ-SAC has the cheapest Gaussian actor but lacks the multimodal generative policy class. DIME, QSM and QVPO sit in a higher-latency region because they evaluate iterative diffusion-style actors. SoftGAC occupies a favorable region across the hard tasks because its actor remains close to one-pass inference cost while achieving higher or competitive IQM return.

Training wall-clock time includes environment interaction and depends on host load, system scheduling and run placement. We therefore report the median elapsed time across seeds at each environment step, alongside the isolated actor-latency measurements. SoftGAC reaches high returns within a wall-clock budget comparable to low-NFE flow baselines, while DIME and QVPO incur the cost of repeated diffusion-network evaluations.

Actor-width sensitivity varies across tasks. Increasing width generally improves or stabilizes learning on Humanoid Run and H1 Hurdle. On Dog Run, the 128-wide bridge already performs well with

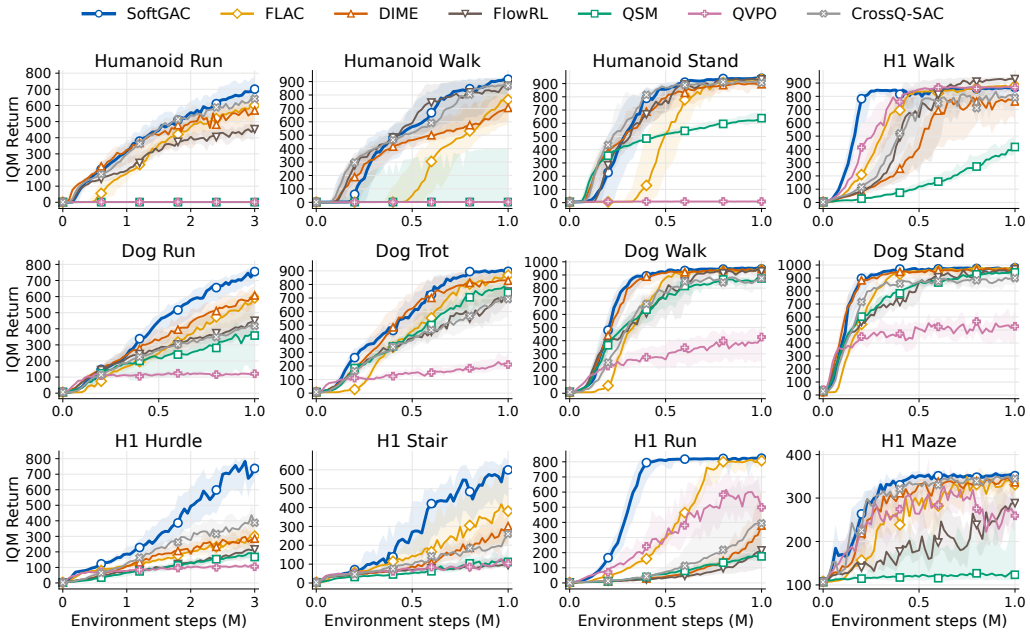


Figure 8: Full IQM learning curves on all benchmark tasks.

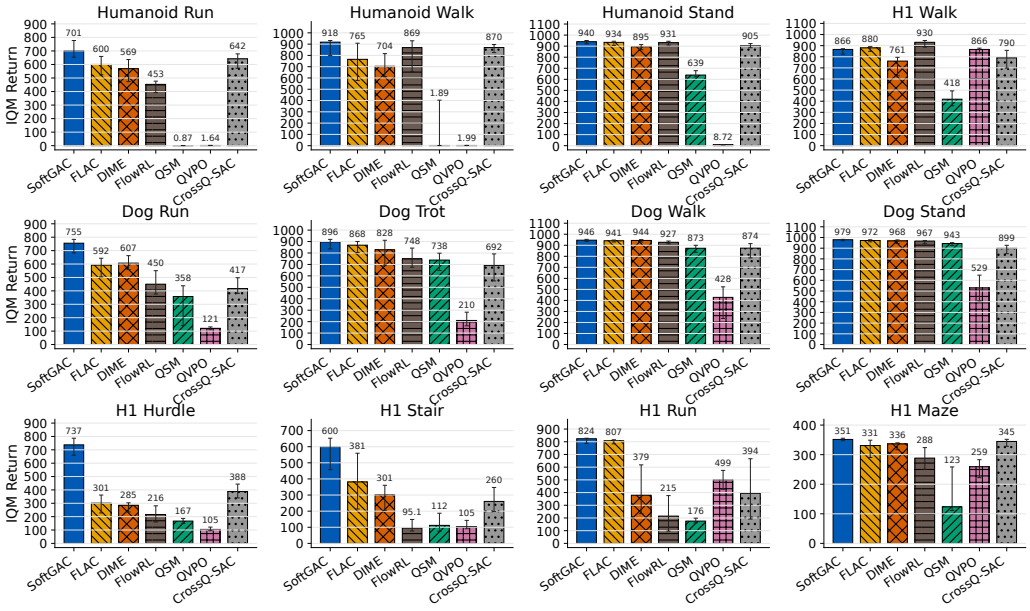


Figure 9: Final IQM return by task.

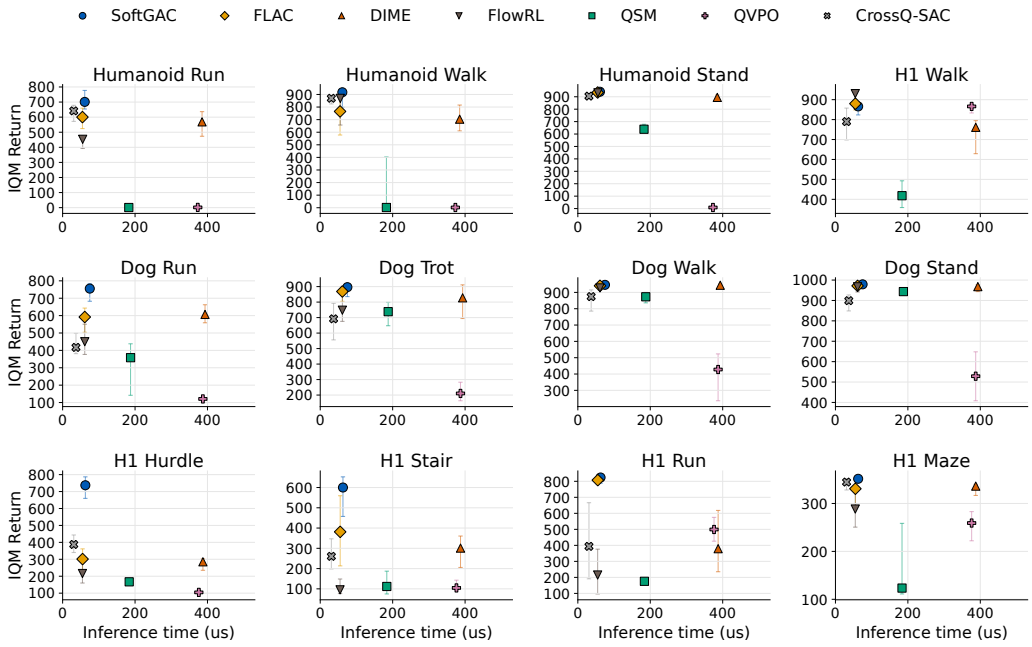


Figure 10: Compute-return tradeoff by task. The horizontal axis is per-action actor inference time, and the vertical axis is IQM return.

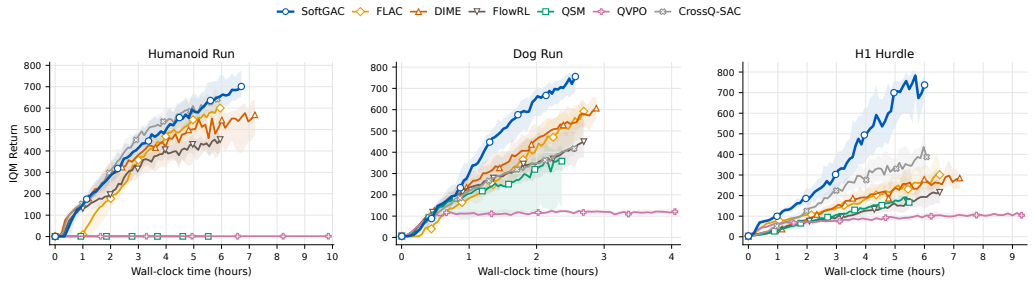


Figure 11: Wall-clock IQM learning curves on Humanoid Run, Dog Run and H1 Hurdle. The horizontal axis uses median seed wall-clock time at each evaluation step. All runs use the v6e-8 TPU VM, but the actual time depends on system scheduling and load.

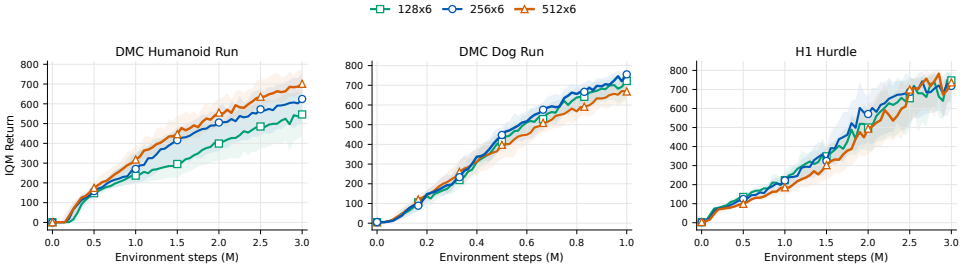


Figure 12: Actor-width sensitivity on Humanoid Run, Dog Run and H1 Hurdle.

fewer parameters, while wider actors do not provide a consistent gain. We use width 512 outside the Dog domain and width 256 for Dog to keep actor parameter counts comparable to the baselines.

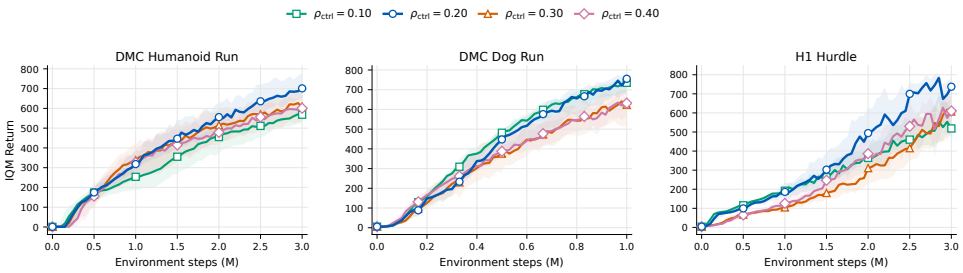


Figure 13: Target control-budget sensitivity on Humanoid Run, Dog Run and H1 Hurdle.

The control-budget sweep reveals a tradeoff between reference matching and value-guided concentration. Small ρ_{ctrl} keeps the bridge close to the high-entropy reference and can restrict concentration on high-value actions. Larger budgets permit stronger control but do not always improve returns. The default $\rho_{\text{ctrl}} = 0.20$ gives stable performance across these three domains without task-specific tuning, consistent with the scale analysis in Appendix E.4.

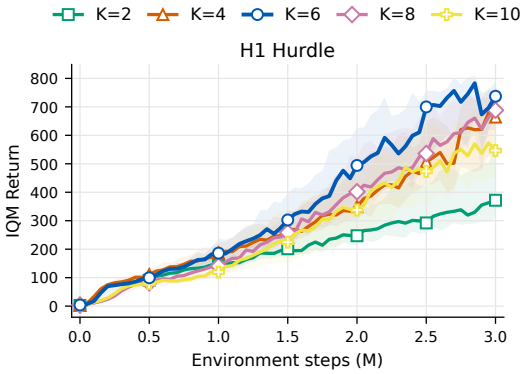


Figure 14: Bridge-depth sensitivity on H1 Hurdle with $K \in \{2, 4, 6, 8, 10\}$.

Figure 14 varies the number of local transition blocks under the default schedule $\mathcal{C}_{\text{target}} = 0.2Kd_a$, so the sweep measures joint sensitivity to depth and total control budget. Under this schedule, $K = 6$ gives the best H1 Hurdle result. The ablation in Appendix F.7 varies depth at a fixed total budget and reports the realized path KL. Together, the two experiments support $K = 6$ as a robust default, without a monotonic relationship between depth and return.

F.2 Conditional Action Multimodality

We test whether the learned state-conditional action laws exhibit multimodality using projection pursuit with Hartigan's Dip statistic (Krause and Liebscher, 2005; Kolyvakis and Likas, 2025). For each task-actor pair, we select the highest-return seed and collect 1000 rollout states. At every state, we sample 8192 actions and split them into 4096 discovery samples and 4096 held-out validation samples. The discovery split searches coordinate axes, fixed random directions reused across states and projection-pursuit directions. We then apply the Dip test only to the selected one-dimensional projection on the held-out split. A state is counted as multimodal when $p < 0.05$.

Table 5: State-conditional action multimodality. IQM returns aggregate eight training seeds. Multimodality percentages are computed over 1000 rollout states from the highest-return seed.

Task	d_a	Actor	IQM return	Multimodal states
H1 Hurdle	19	`SoftGAC`	737	53.8%
		CrossQ-SAC	388	0.0%
H1 Run	19	`SoftGAC`	824	18.6%
		CrossQ-SAC	394	0.0%
Walker Walk	6	`SoftGAC`	980	0.2%
		CrossQ-SAC	981	0.0%

The test detects substantial projected multimodality for `SoftGAC` on both H1 tasks and none for CrossQ-SAC. On Walker Walk, where both methods achieve near-saturated returns, `SoftGAC` rarely produces a significantly multimodal projection. The observed multimodality thus varies with the task rather than appearing uniformly across the bridge policies.

F.3 Path Regularizer and Base Distribution

To isolate the choice of regularizer, we replace the path KL with the GMM endpoint entropy estimator used by DACER. The estimator fits a three-component GMM to 200 sampled endpoint actions, and we use DACER's target-entropy setting. The actor architecture, critic and training budget remain fixed. In a separate ablation, we replace the logistic base with an isotropic standard Gaussian. The full path KL then additionally contains the fixed initial-law term $D_{\mathrm{KL}}(p_0^{\mathrm{Gaussian}} \| r_0^{\mathrm{logistic}})$.

Table 6: Path-regularizer and base-distribution ablations. Per-task values are IQM $\pm$ standard deviation over eight seeds.

(a) Analytical path KL versus GMM endpoint entropy		
Task	Path-KL regularizer	GMM entropy estimator
Dog Run (1M)	755 ± 64	586 ± 77
H1 Hurdle (3M)	737 ± 107	347 ± 152
H1 Run (1M)	824 ± 32	192 ± 57
H1 Stair (1M)	600 ± 125	270 ± 155
Average	729	349
(b) Logistic versus standard-Gaussian base		
Task	Logistic base	Gaussian base
H1 Hurdle (3M)	737 ± 107	670 ± 175
H1 Run (1M)	824 ± 32	823 ± 14
H1 Stair (1M)	600 ± 125	562 ± 136
Average	720	685

The path-KL regularizer yields higher IQM returns on all four tasks. The GMM approach replaces the analytical path regularizer with a finite-sample endpoint-density approximation and adds the cost of action sampling and mixture fitting. Differences between the two base distributions are smaller. The logistic base gives slightly higher average return, but the differences are small relative to run-to-run variability, indicating robustness to these two base choices. We use the logistic base because its tanh image is uniform, which simplifies the reference construction.

F.4 Diffusion and Normalizing-Flow Baselines

We compare `SoftGAC` with DACER, MaxEntDP and SDAC under the same C51+CrossQ main critic, preserving each baseline's actor objective, entropy treatment and auxiliary components. Actor parameter counts are approximately matched, and all three diffusion baselines use a shared denoiser with 20 sampling steps. We evaluate all four methods on Walker Run, H1 Run and H1 Hurdle.

Table 7: Comparison with recent diffusion-policy baselines. Values are IQM $\pm$ standard deviation over eight seeds.

Actor	Walker Run (1M)	H1 Run (1M)	H1 Hurdle (3M)	Average
SoftGAC	823 ± 21	824 ± 32	737 ± 107	795
DACER	860 ± 32	537 ± 238	377 ± 75	591
MaxEntDP	629 ± 58	23 ± 8	18 ± 3	223
SDAC	836 ± 34	36 ± 5	60 ± 24	311

All four methods learn on Walker Run, but their returns differ markedly on the harder H1 tasks. MaxEntDP uses Monte Carlo approximations in both its actor update and entropy-aware critic target, while SDAC trains a diffusion actor with reweighted score matching. SoftGAC avoids endpoint-density estimation and backpropagation through a repeated denoiser, and achieves the highest average IQM across these three tasks.

We also adapt a conditional normalizing-flow policy (Ghugare and Eysenbach, 2025) to online SAC. It retains exact squashed-action log density and its flow architecture, while sharing the same main critic and training budget as the other methods.

Table 8: Comparison with DACER and a conditional normalizing-flow actor. Values are IQM $\pm$ standard deviation over eight seeds.

Actor	Dog Run (1M)	H1 Hurdle (3M)	H1 Run (1M)	H1 Stair (1M)	Humanoid Run (3M)	Average
SoftGAC	755 ± 64	737 ± 107	824 ± 32	600 ± 125	701 ± 64	724
DACER	542 ± 76	377 ± 75	537 ± 238	164 ± 102	541 ± 82	432
Normalizing Flow	503 ± 95	325 ± 79	590 ± 234	376 ± 131	597 ± 73	478

A normalizing flow makes endpoint density tractable by requiring every transformation to be bijective with a tractable Jacobian determinant. This dimension-preserving structure constrains its architecture. The stochastic bridge blocks in SoftGAC need neither invertibility nor tractable Jacobians because local transition densities already expose the analytical path KL. SoftGAC is strongest on every task in Table 8.

F.5 Actor Cost and Training Wall-Clock Time

Table 9 reports evaluation-time actor parameters and single-action latency on an Apple M3 Pro CPU, excluding compilation. During evaluation, SoftGAC follows its deterministic mean path and bypasses variance heads, DACER omits GMM fitting and the normalizing flow omits log-determinant computation.

Table 9: Evaluation-only actor parameters and inference latency by domain.

Actor	NFE	DMC Dog params / time	HumanoidBench params / time	DMC Humanoid params / time
SoftGAC	1	0.526M / 75 μs	0.342M / 63 μs	0.410M / 61 μs
DACER	20	0.426M / 414 μs	0.318M / 400 μs	0.328M / 404 μs
Normalizing Flow	1	0.413M / 129 μs	0.344M / 97 μs	0.351M / 98 μs

Table 10: Actor cost and median training wall-clock time on H1 Hurdle at 3M environment steps. Actor parameters count the complete trainable actor. Training uses one TPU core per seed.

Actor	NFE	Actor params	Inference	Training time
SoftGAC	1	0.342M	63 μs	6.0 h
DACER	20	0.318M	400 μs	7.6 h
MaxEntDP	20	0.399M	509 μs	24.9 h
SDAC	20	0.318M	410 μs	9.3 h

DACER is 5.5–6.6$\times$ slower than SoftGAC at deployment across the three domains, while the one-pass normalizing flow is 1.5–1.7$\times$ slower. On H1 Hurdle, SoftGAC also trains faster than

all three diffusion baselines. MaxEntDP is especially costly because its actor update and entropy approximation use extensive Monte Carlo sampling.

F.6 Transfer to an XQC Critic Backend

To test sensitivity to the critic architecture and update rule, we train `SoftGAC`, DIME and SAC with the same XQC critic (Palenicek et al., 2026). Unlike the C51+CrossQ configuration, XQC uses an exponential-moving-average target critic. We keep the actor objectives, actor architectures and training budgets unchanged.

Table 11: Actor comparison after replacing the shared critic backend with XQC. Values are IQM $\pm$ standard deviation over eight seeds.

Actor	Dog Run (1M)	H1 Slide (1M)	H1 Stair (1M)	Average
`SoftGAC`	666 ± 64	829 ± 160	447 ± 124	647
DIME	483 ± 45	663 ± 115	298 ± 101	482
SAC	486 ± 47	478 ± 121	288 ± 115	417

`SoftGAC` achieves the highest IQM on all three tasks with the XQC critic. Its average IQM is 34% higher than DIME and 55% higher than SAC. The advantage therefore persists beyond the C51+CrossQ critic used in the other comparisons.

F.7 Learned Variance and Fixed-Budget Bridge Depth

The local Gaussian path KL contains both drift and variance terms. We isolate them on H1 Stair by fixing the transition variance σ^2 throughout training. The setting $\sigma^2 = 1$ matches the reference variance, makes the variance KL zero and leaves only the drift cost.

Table 12: Transition-variance ablation on H1 Stair at 1M steps. Values are IQM $\pm$ standard deviation over eight seeds.

Variance during training	H1 Stair IQM
Learned variance (default)	600 ± 125
Fixed $\sigma^2 = 2.0$	42 ± 4
Fixed $\sigma^2 = 1.0$ (drift-only KL)	529 ± 126
Fixed $\sigma^2 = 0.5$	243 ± 172
Fixed $\sigma^2 = 0.1$	8 ± 0

The learned variance is strongest, while the drift-only variant remains effective. Fixed variances above or below the reference value degrade learning, which supports adaptive transition-scale exploration during training. The variance heads are bypassed during deterministic evaluation.

To isolate bridge depth from the target KL budget, we evaluate Dog Run with $\mathcal{C}_{\text{target}} = 0.2 \times 6 \times d_a = 45.6$ for every K, where $d_a = 38$. Equivalently, $\rho_{\text{ctrl}} = 1.2/K$. The hidden width remains 256.

Table 13: Bridge depth on Dog Run under a fixed total control budget. Return values report IQM $\pm$ standard deviation across eight seeds.

Bridge depth K	Actor params	Empirical path KL	Dog Run IQM
4	351K	45.6 ± 0.1	746 ± 62
5	438K	45.7 ± 0.1	716 ± 56
6 (default)	526K	45.6 ± 0.1	755 ± 64
7	613K	45.6 ± 0.1	704 ± 99
8	701K	45.6 ± 0.2	711 ± 78

The realized KL remains at the common target and performance has no monotonic trend over $K = 4$ to 8. The method is therefore insensitive to moderate depth changes under a fixed budget on this task. Scaling to deeper bridge networks may require architecture-specific optimization, which we leave for future work.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the proposed path-space soft generative actor-critic method, the low-cost bridge-policy design and the empirical compute-return claims. The scope is explicitly limited to state-based simulated online continuous control, and the paper includes matching theoretical analysis, finite-reference discussion, ablations and experiments.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The final section discusses the finite-step Gaussian bridge, reference and base-law choices, added implementation complexity, bridge hyperparameters and the current restriction to state-based simulated control. It identifies image-conditioned learning, world models and real-robot evaluation as future work. Appendices D and F quantify the relevant biases and robustness.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best

judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: The main text states the assumptions for each proposition, theorem, corollary and lemma, and Appendix A restates each result before giving the proof. Appendix D separately analyzes the finite-step reference endpoint bias used by the implementation.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Appendix E provides benchmark tasks, training hardware, implementation details, the control-energy budget choice, hyperparameters, parameter counts and optimizer settings. Appendix B gives pseudocode for the training loop. Appendix F states the protocols for the multimodality, baseline, critic-backend and ablation experiments.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

(c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The implementation and experiment scripts are available in the repository linked below the abstract. Appendix E describes the environments, implementation details, hyperparameters and compute resources. The experiments use standard simulator benchmarks rather than a new dataset.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Appendix E.1 reports the main environments, observation dimensions and action dimensions. Appendices E.2, E.3, E.4 and E.5 report hardware, implementation details, the control-energy budget heuristic, optimizer settings, main hyperparameters, actor and critic sizes and parameter counts. Appendix F gives protocols and settings for the extended evaluations.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: The main experiments report IQM over 8 random seeds with 95% bootstrap confidence intervals. These intervals are shown in the main figures and Appendix F. Extended return tables in the same appendix report IQM $\pm$ standard deviation over eight seeds and identify the one-seed multimodality analysis separately.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Appendix E.2 reports the cloud TPU type, CPU model, CPU count, memory and operating system of the training machines. Appendices F and F.5 report representative wall-clock curves, actor latency and training time for difficult tasks.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work studies an algorithmic reinforcement-learning method in simulated control benchmarks. It does not involve human subjects, private data, web scraping, safety-critical deployment or personally identifiable information.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: This is foundational algorithmic work evaluated only in simulated continuous-control benchmarks. We do not identify a direct societal deployment path or direct societal impact beyond general advances in reinforcement learning.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release high-risk pretrained models, image generators, scraped datasets or user-facing systems. The experiments use simulated control environments.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.

- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: Appendix E.1 lists the benchmark assets used for evaluation, gives URLs for DeepMind Control Suite and HumanoidBench, and states their licenses. Table 2 lists the official baseline repositories used as implementation references, and the baseline methods are credited through their cited papers.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The accompanying repository provides implementations of the proposed method and baselines. The repository contains experiment scripts, configuration files and instructions needed to run the reported methods. The paper does not introduce a new dataset, benchmark suite or pretrained model asset.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The work does not involve crowdsourcing or research with human subjects.

    Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The work does not involve crowdsourcing or research with human subjects, so IRB approval is not applicable.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs are not part of the proposed method, experiments, baselines, theoretical analysis or evaluation protocol.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.